COMPLEXITY THEORY · FOUNDATIONS FIRST · 27 OCTOBER 2026

Beyond exponential search

What would it mean to refute the Exponential Time Hypothesis? Start with a puzzle, learn the language of computational difficulty, and follow the decisive ideas in OpenAI's paper.

You know some Big O notation. No knowledge of logic, complexity classes, coding theory, tensor networks, or proof assistants is assumed. The elementary arguments are developed here. The research argument is an annotated reading guide with explicit boundaries, rather than a substitute for its 67-page proof.

Foundations route
Read in order. Stop at the worked examples and solve a few conceptual problems.

Aha route
Visit the paper's turning points, then follow the prerequisite links when a step is unfamiliar.

STATUS OF THE CENTRAL CLAIM

The supplied manuscript states a deterministic, exact counting algorithm with subexponential running time and polynomial space. OpenAI's announcement reports Lean formalization and an external group organized to examine the result. This tutorial explains the claim and its mathematical implications; it does not certify the complete research proof or report a completed independent consensus review. The repository's scope note covers a weaker exponent than the sharpest revised manuscript bound. The distinction matters.

Primary status sources: OpenAI announcement, 26 October 2026; Lean scope note; paper, Theorem 1.1, p. 2. Checked 27 October 2026.

The Aha trail

Follow these links to the highlighted turning points. The annotated paper map supplies the corresponding theorem, equation, and page references.

1. A large candidate set is not a lower bound.
2. Polynomial reductions can destroy a subexponential saving.
3. Refuting ETH leaves P versus NP unsettled.
4. Count solutions without listing them.
5. Separate groups by fixing a small interface.
6. Taylor order limits the corrections to track.
7. Low degree and spectral concentration fit together.
8. The qualitative refutation is complete before the sharpest bound.
9. Counting constructs the parity filter.
10. Divisibility removes interactions; recursion shrinks what remains.

Visual explanations: CNF circuit · growth rates · shared summaries · Hermite recovery · spectral window · parity filtering · tensor network · why the claimed bound refutes ETH.

1. A puzzle becomes a Boolean formula

Suppose three choices must obey three rules. At least one of choices x and y must be made. If x is made, z must be made. Choices y and z cannot both be made. Does any combination obey all three rules? How many combinations do?

A *Boolean variable* takes one of two values: 0 for false and 1 for true. A *Boolean assignment* specifies a value for every variable. The three choices therefore have eight possible assignments. More generally, n Boolean variables have 2^n assignments because each of the n independent positions has two choices. Here n is the number of variables.

> **THE LOGIC OPERATIONS**
>
> Negation, written $\neg x$, reverses truth. Disjunction, written $x \vee y$, means at least one input is true, including the case where both are true. Conjunction, written $x \wedge y$, means both inputs are true. On bits, $\neg x = 1 - x$, $x \wedge y = xy$, and $x \vee y = 1 - (1 - x)(1 - y)$.

A *literal* is a variable or its negation. A *clause* is a disjunction of literals. A formula is in *conjunctive normal form*, abbreviated CNF, when it is a conjunction of clauses. All clauses must hold; within each clause, one true literal is enough. Our running formula is

$$F(x, y, z) = (x \vee y) \wedge (\neg x \vee z) \wedge (\neg y \vee \neg z).$$

The expression $\neg x \vee z$ encodes the implication from x to z: it fails only when $x = 1$ and $z = 0$. The last clause fails only when $y = z = 1$. The following table evaluates every possibility.

Running example. Each row is an assignment; $F = 1$ means all rules hold.

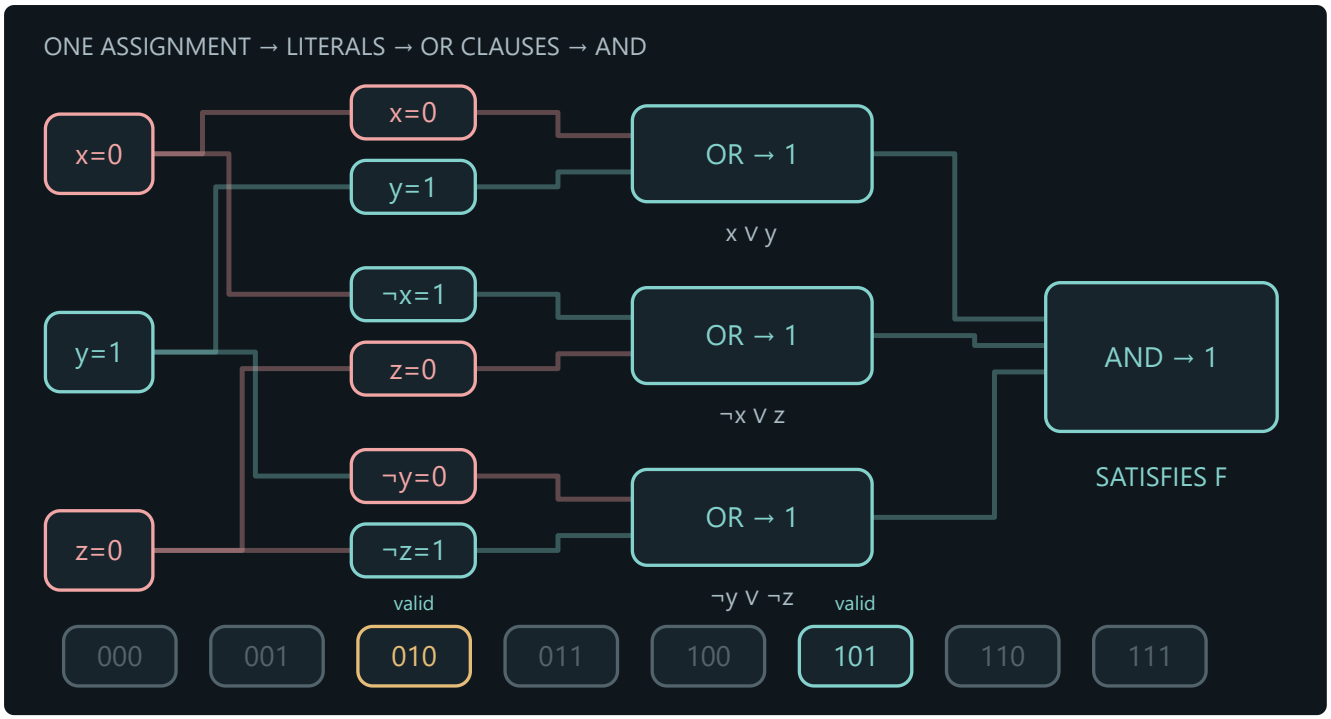
x	y	z	$x \vee y$	$\neg x \vee z$	$\neg y \vee \neg z$	F

0	0	0	0	1	1	0
0	0	1	0	1	1	0
0	1	0	1	1	1	**1**
0	1	1	1	1	0	0
1	0	0	1	0	1	0
1	0	1	1	1	1	**1**
1	1	0	1	0	1	0
1	1	1	1	1	0	0

The *satisfiability problem*, or SAT, asks whether a formula has at least one satisfying assignment. This is a *decision problem*: its answer is yes or no. The *search problem* asks for a satisfying assignment. The *counting problem*, written #SAT, asks for the exact number. Our formula has decision answer yes, search answers $(0, 1, 0)$ and $(1, 0, 1)$, and count two.

A logic gate computes one Boolean operation. Fig. 1 turns the formula into a circuit: negation produces the negative literals, each OR gate evaluates one clause, and the final AND gate requires every clause to pass. Each wire carries a bit. Crossings without a junction do not connect wires.

Watch a CNF circuit evaluate an assignment



Assignment 010: clause values 1, 1, 1; F=1. Only 010 and 101 satisfy the formula. One circuit evaluation checks one assignment; it is not the subexponential solver.

Fig. 1. A finite circuit checks one assignment. Stepping through all eight reproduces the truth table; it does not implement the paper's counting algorithm. Teal means 1 and pink means 0, with explicit bit labels. Motion shows evaluation changes, not physical gate delays.

A formula is a k-*CNF* when every clause has at most k literals. Its *width* is the largest clause length. The problem restricted to such formulas is k-SAT. Our formula is a 2-CNF and is also permitted as a 3-CNF. Using a small 2-CNF makes the arithmetic transparent; it does not illustrate the worst-case difficulty of 3-SAT. The decision problem 2-SAT has a polynomial-time algorithm, while exact #2-SAT counting is a different task.

Two special conventions prevent counting errors. An empty clause is false, so any formula containing one is unsatisfiable. A conjunction of no clauses is true for every assignment, giving 2^n solutions on n declared variables. An unused variable still contributes a factor of two. Declaring a fourth unused variable in our example would change the count from two to four.

AHA 1 · A LARGE CANDIDATE SET IS NOT A LOWER BOUND

Eight rows describe the possible assignments. They do not establish that every method must inspect eight rows. For n variables, a formula requiring every variable to be true still has 2^n possible assignments and an immediate unique solution. The difficult question is what one algorithm can guarantee on *every* input formula.

Background: Aaronson, pp. 12–15; warning against counting candidates as a lower-bound proof: pp. 29–31. The running formula and table are original teaching constructions.

2. The gap between polynomial and exponential

An algorithm is a finite procedure that accepts a specified input and eventually produces the required answer. Its *worst-case running time* at a given input size is the largest number of computation steps needed for any input of that size. A fast answer on one formula gives no worst-case bound for all formulas.

Big O describes an eventual upper bound. For nonnegative functions f and g, the statement $f(n) = O(g(n))$ means there are constants $C > 0$ and n_0 such that $f(n) \leq Cg(n)$ for all integers $n \geq n_0$. The constants do not change with n. Big Omega, $f(n) = \Omega(g(n))$, means an eventual lower bound of the same kind. Big Theta, Θ, means both bounds hold.

Little o is more demanding than Big O. The statement $f(n) = o(g(n))$ means $f(n)/g(n)$ tends to zero as n grows. Equivalently, for each constant $\epsilon > 0$, the inequality $f(n) \leq \epsilon g(n)$ holds for all sufficiently large n. That threshold may depend on ϵ. Thus $\sqrt{n} = o(n)$, but $0.01n$ is not $o(n)$.

> **THE GROWTH CLASSES USED HERE**
>
> **Polynomial time** has a bound $n^{O(1)}$, such as n^3 or n^{100}. **Single-exponential time** has a bound $2^{O(n)}$; a function 2^{cn} with fixed $c > 0$ is exponential even if c is small. **Subexponential time in n** means $2^{o(n)}$: the exponent divided by n tends to zero. Polynomial time is included in subexponential time because $n^a = 2^{a \log_2 n}$ and $\log_2 n = o(n)$ for every fixed a.

Here $\log_2 n$ is the base-two logarithm: the exponent b satisfying $2^b = n$. An undecorated $\log n$ below means the natural logarithm, as in most of the OpenAI paper. Changing between fixed logarithm bases changes constants in these asymptotic classifications.

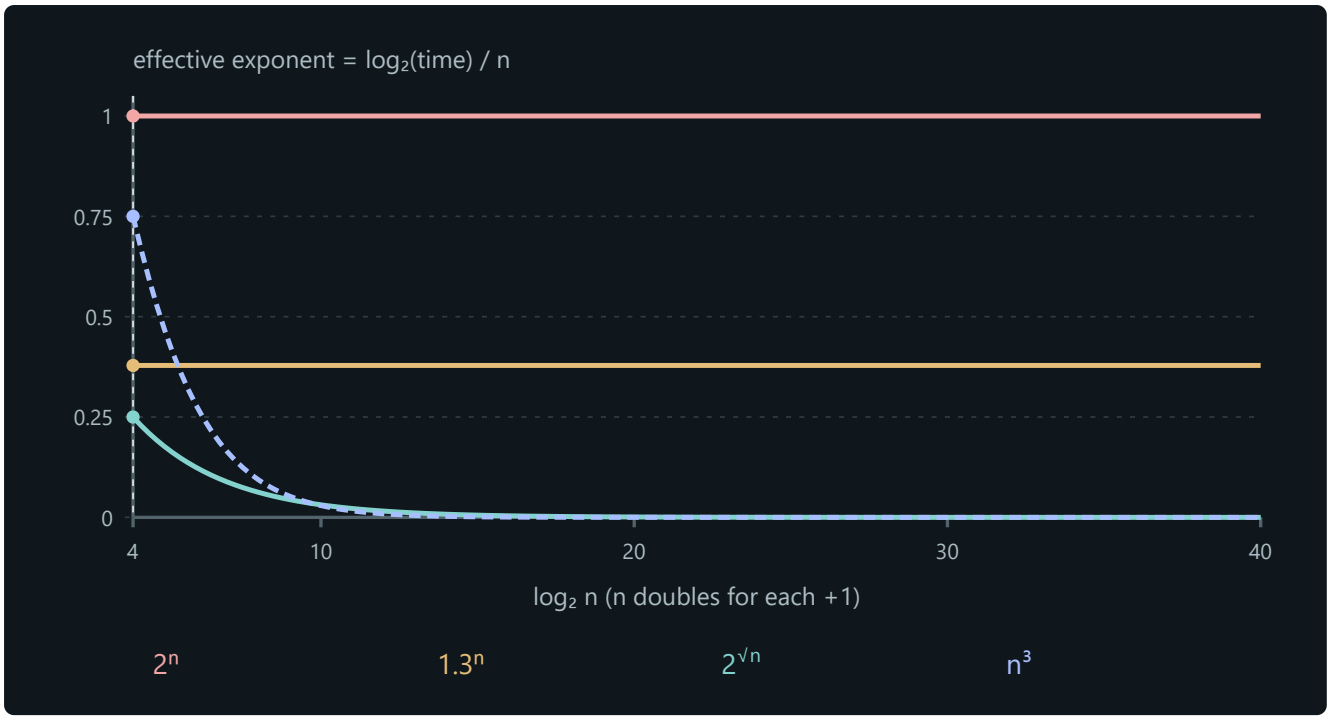
Time function	Exponent when written as a power of two	Interpretation
n^3	$3 \log_2 n$	Polynomial, hence subexponential.
$n^{\log_2 n}$	$(\log_2 n)^2$	Quasipolynomial: above every fixed polynomial eventually, but subexponential.
$2^{\sqrt{n}}$	$\sqrt{n}$	Subexponential, and this particular function is superpolynomial.
$2^{n/\log n}$	$n/\log n$	Subexponential despite an enormous eventual running time.
1.3^n	$n \log_2(1.3) \approx 0.378512n$	A constant-factor reduction in the exponent; still exponential.
2^n	n	Exhaustive Boolean enumeration, apart from checking costs.

A COMPARISON THAT AVOIDS HUGE INTEGERS

Compare base-two logarithms of time rather than the time itself. At $n = 1024$, the functions n^3, $2^{\sqrt{n}}$, 1.3^n, and 2^n have log-times 30, 32, approximately 387.596, and 1024, respectively. These are illustrative functions with coefficient one, not runtime estimates for the OpenAI algorithm. An expression hiding unknown constants cannot provide a practical crossover point.

Fig. 2 divides each log-time by n. This exposes the distinction between reducing an exponential rate to a smaller positive constant and making the rate tend to zero.

Which exponent rates approach zero?



n=2^4: exponent ratios are 1 for 2^n, 0.3785 for 1.3^n, 0.250 for $2^{(\sqrt{n})}$, and 0.750 for n^3. The last two tend to zero; a smaller fixed positive rate is still exponential.

Fig. 2. The horizontal axis is $\log_2 n$; the vertical axis is $\log_2(\text{time})/n$. Fixed positive horizontal rates remain exponential. A rate approaching zero is subexponential. These are illustrative functions, not predicted runtimes or crossover estimates for OpenAI's algorithm.

Variables are not the entire input

A SAT formula has n variables and m clauses. Let L denote the number of bits in an explicit encoding of the formula. Writing a variable index takes about $\log_2 n$ bits, so a fixed-width formula can be encoded in polynomially related quantities such as $n + m$ and L, with details depending on the convention. Reading a very long input already costs time. The paper therefore retains a polynomial input-size factor, rather than claiming a bound depending only on n for arbitrarily large m.

The notation $O_k(1)$ means a constant that may depend on k. "For each fixed k," means choosing a clause-width bound first and then letting n grow. It does not supply the same constants when k itself grows with n. This distinction will limit the consequences for unrestricted CNF-SAT.

> **A CALCULATION PROVED HERE**
>
> For every fixed nonnegative constant a, the function

$$g_a(n) = \frac{n(\log\log n)^a}{\sqrt{\log n}}$$

is $o(n)$ as $n \to \infty$. Consequently, $2^{O(g_a(n))}$ is subexponential in n.

Divide by n, and put $t = \log n$. The ratio becomes $(\log t)^a/\sqrt{t}$. Next put $u = \log t$, obtaining $u^a e^{-u/2}$. The exponential series gives $e^{u/2} \geq (u/2)^b/b!$ for any fixed integer $b > a$ and positive u. Hence the ratio is at most $b!2^b u^{a-b}$, which tends to zero. Multiplying this ratio by any fixed constant still gives zero. This proves the claim. ■

A polynomial input-size factor contributes only $O(\log n)$ to the exponent when input size is polynomial in n. For fixed k, duplicate and tautological clauses can be removed, leaving only polynomially many distinct width-at-most-k clauses; the original input-reading cost remains explicit. This is why the paper's stated bounds have the form $\mathrm{poly}(n, m)2^{o(n)}$.

An upper bound is not an optimality theorem. If an algorithm is bounded by $2^{\sqrt{n}}$, that does not prove the problem lacks a polynomial-time algorithm. Even that algorithm might admit a sharper analysis. Distinguish "a superpolynomial function occurs in the proven bound" from "the problem requires superpolynomial time."

Notation background: Aaronson, p. 22. Asymptotic versus practical efficiency: pp. 6–7. Paper conventions: Section 1.2, p. 7.

3. What P versus NP actually asks

Given the assignment $(0, 1, 0)$, checking our formula requires evaluating three short clauses. Discovering an assignment without being given one can be harder. The distinction between checking a proposed solution and finding one motivates two complexity classes.

> **DECISION CLASSES**
>
> $\mathbf{P}$ is the class of decision problems solvable by a deterministic algorithm in time polynomial in input bit length L. Deterministic means each next step is fixed by the current computation state and input.
>
> $\mathbf{NP}$ is the class of decision problems for which every yes-instance has a certificate of polynomial length that a deterministic algorithm can verify in polynomial time. No certificate may convince the verifier to accept a no-instance. "NP" abbreviates nondeterministic polynomial time; it does not mean non-polynomial.

A satisfying assignment is a certificate for SAT. It contains n bits. Checking all literals takes time polynomial in the explicit formula length. If the formula has no satisfying assignment, no assignment passes the check. These two properties establish SAT's membership in NP. They do not establish that finding a certificate is difficult.

Every problem in P is also in NP: a verifier can ignore a proposed certificate and run the polynomial-time decision algorithm. Thus $\mathbf{P} \subseteq \mathbf{NP}$. The question $\mathbf{P} = \mathbf{NP}$ asks whether the converse inclusion holds. In plain English: can every decision problem with short, efficiently checkable yes-certificates also be decided efficiently?

Reductions transfer algorithms in one direction

A *polynomial-time many-one reduction* from decision problem A to decision problem B is a transformation R computable in polynomial time such that input u is a yes-instance of A exactly when $R(u)$ is a yes-instance of B. An algorithm for B then solves A: transform, solve, return the answer.

A decision problem is *NP-hard* if every NP problem has such a reduction to it. It is *NP-complete* if it is NP-hard and also belongs to NP. The Cook–Levin theorem, followed by the standard bounded-width conversion, establishes that 3-SAT is NP-complete. The full general theorem is cited here, not proved. Its consequence is precise: a polynomial-time algorithm for 3-SAT would give a polynomial-time algorithm for every NP decision problem.

ONE GATE, ENCODED EXACTLY

A Boolean circuit combines small operations, or gates. To encode an AND gate with input bits u, v and output bit w, use

$$(\neg w \vee u) \wedge (\neg w \vee v) \wedge (w \vee \neg u \vee \neg v).$$

If $w = 1$, the first two clauses force $u = v = 1$. If $u = v = 1$, the third forces $w = 1$. If either input is zero, the corresponding first or second clause forces $w = 0$, and the third then holds. The clauses therefore enforce exactly $w = uv$. Every input pair has exactly one allowed output bit.

Encoding every gate and requiring the final output to be one converts circuit satisfiability to 3-SAT. Because internal outputs are determined uniquely, this particular encoding also preserves the number of satisfying input assignments. Such a *parsimonious reduction* preserves counts, a stronger condition than preserving yes/no answers.

Decision, search, and counting

A SAT decision algorithm can find a witness with at most one test per variable, after an initial satisfiability test. Ask whether there is a solution with the first variable zero. If yes, keep zero; otherwise keep one. Repeat under the choices already fixed. The invariant is that the current partial assignment extends to a full solution. After n choices it is a solution. This is *self-reduction*: solving a problem by asking smaller instances of itself.

A counting algorithm is stronger in an immediate way: compute the count and test whether it exceeds zero. The reverse implication is not immediate. Recursing down both branches of every variable to add their counts may revisit an exponential number of subproblems. Polynomial-time SAT decision would not, by this argument alone, give polynomial-time exact #SAT counting.

AHA 2 · POLYNOMIAL REDUCTIONS ARE TOO COARSE FOR ETH

Suppose a reduction turns a problem of size N into SAT with $n = N^3$ variables. A $2^{\sqrt{n}}$ SAT algorithm becomes $2^{N^{3/2}}$. That is not subexponential in N. Polynomial time survives polynomial-size reductions because $(N^3)^c = N^{3c}$. Subexponential-in-variables guarantees need much tighter size control.

This is the central habit of *fine-grained complexity*: track how a reduction changes the exact parameter in the exponent, rather than keeping only the word "polynomial." A reduction from SAT to a target problem establishes conditional hardness of the target. A new SAT algorithm does not run that arrow backward; a target-to-SAT reduction with suitable size bounds is needed to obtain a new target algorithm.

Definitions and NP-completeness: Aaronson, Section 2, pp. 10–16. Search-to-decision: p. 16. Counting: p. 21. Reduction-size warning in the OpenAI manuscript: p. 2 and Section 1.1.3, p. 5.

4. ETH names a much higher barrier than P ≠ NP

The claim $\mathbf{P \neq NP}$ rules out polynomial-time algorithms for 3-SAT. It leaves open an enormous range of possible running times above polynomial and below ordinary exponential. The Exponential Time Hypothesis makes a stronger prediction about that range.

> **EXPONENTIAL TIME HYPOTHESIS · ETH**
>
> In the deterministic setting used here, ETH asserts a positive exponential rate for worst-case 3-SAT in its number n of variables, with polynomial input-size factors understood. It rules out a deterministic $2^{o(n)}\,\mathrm{poly}(n, m)$ algorithm for arbitrary 3-CNFs. The standard positive-rate formulation says there is a constant $\delta > 0$ such that a $2^{\delta n}\,\mathrm{poly}(n, m)$ algorithm is impossible. For refuting it, the explicit $2^{o(n)}$ algorithm claimed by this paper suffices under either formulation.

ETH never said brute force was optimal. A 1.3^n algorithm has time $2^{0.378512\ldots n}$, retaining a positive constant in front of n in its exponent. It would be consistent with ETH. By contrast, $2^{\sqrt{n}}$ has exponent-to-variable ratio $1/\sqrt{n} \to 0$, and would refute ETH if it solved arbitrary 3-SAT with polynomial input overhead.

SETH
Rates for fixed widths approach brute-force rate as width grows.

⇒

ETH
3-SAT retains some positive exponential rate.

⇒

P ≠ NP
No polynomial-time 3-SAT algorithm.

The *Strong Exponential Time Hypothesis*, or SETH, says that for every constant $\epsilon > 0$, some fixed width k has no deterministic $2^{(1-\epsilon)n}\,\mathrm{poly}(n, m)$ algorithm. The ordering of "for every ϵ" and "some fixed k" matters. SETH implies ETH using the standard sparsification connection between fixed-width SAT problems. That implication is a cited result, not a consequence of 3-SAT merely being a subset of k-SAT.

The *counting ETH*, or #ETH, predicts an exponential barrier for exact #3-SAT. Since counting immediately solves decision, ETH implies #ETH. Their converses are not established by that observation. A deterministic subexponential counter for every fixed width would refute both ETH and #ETH, and also SETH. The manuscript discusses further strengthened variants, including gap problems, on p. 4.

Why does ETH imply $\mathbf{P} \neq \mathbf{NP}$? If $\mathbf{P} = \mathbf{NP}$, 3-SAT has a polynomial-time algorithm. For fixed width, normalize away duplicate clauses, retaining the original input-reading cost. The remaining formula has polynomial size in n. Polynomial time is $2^{o(n)}$ by Section 2, contradicting ETH. This proves the implication. It does not prove its converse. ■

AHA 3 · A FALSE HIGHER BARRIER LEAVES THE LOWER BARRIER UNSETTLED

Proving an algorithm with a sublinear exponent eliminates the ETH barrier. It does not establish a polynomial algorithm. The landscape could still contain a large region of superpolynomial difficulty. Equally, a subexponential upper bound does not prove that such a region must remain. P versus NP is not settled in either direction by that upper bound.

A conditional hardness theorem has the form "if ETH, then no algorithm of type B." A refutation of ETH removes the premise supporting this route to the lower bound. It does not establish the existence of algorithm B, and it does not erase a valid logical proof of the conditional. For a concrete parallel, "if the bridge is closed, this route is unavailable" says nothing about whether a different obstacle blocks the route after the bridge opens.

Historical framing and confidence distinctions: Aaronson, Section 5.1, p. 31. Paper's hypotheses and claimed implications: Section 1.1.1, p. 4; fine-grained consequences: p. 5.

5. The paper starts by counting, not guessing

Our formula has two satisfying assignments. Write $\#F$ for this number. Rather than manipulate truth values directly, represent each clause by an ordinary polynomial that equals one on satisfying Boolean inputs and zero on violating ones. This translation is called *arithmetization*: logical conditions become algebraic expressions.

For a positive literal x_v, use $L_{e,v}(X_v) = X_v$; for a negative literal $\neg x_v$, use $L_{e,v}(X_v) = 1 - X_v$. Here e indexes a clause, v indexes a variable in it, and capital X_v denotes a formal polynomial variable. A lowercase $x_v \in \{0, 1\}$ is a value substituted into that variable. Let I_e be the set of variable indices appearing in clause e.

$$\Phi_e((X_v)_{v \in I_e}) = 1 - \prod_{v \in I_e} (1 - L_{e,v}(X_v)).$$

The product equals one exactly when all literals are false, and is zero otherwise on Boolean inputs. Subtracting from one therefore gives the clause's satisfaction indicator. The symbol $\prod$ means multiply one factor for each indicated index. Similarly, $\sum$ means add.

OUR FORMULA IN ALGEBRA

Its three clause polynomials are

$$\Phi_1 = 1 - (1 - X)(1 - Y), \quad \Phi_2 = 1 - X(1 - Z), \quad \Phi_3 = 1 - YZ.$$

At $(x, y, z) = (0, 1, 0)$, all three values are one and their product is one. At $(1, 1, 1)$, the last value is zero and the product is zero. Every row in the truth table behaves this way. Consequently,

$$\#F = \sum_{(x,y,z)\in\{0,1\}^3} \Phi_1(x, y)\Phi_2(x, z)\Phi_3(y, z) = 2.$$

The notation $\{0, 1\}^3$ means the set of three-bit assignments. The same identity holds for n variables and any number of clauses.

This expression is exact, but writing an exponential sum does not evaluate it quickly. Expanding the product can also create exponentially many terms. The algorithm must find a representation that can be evaluated without listing all global assignments or all expanded monomials.

Why degree becomes an algorithmic resource

The *degree* of a monomial is the sum of its variable exponents. The degree of a polynomial is the maximum degree of its nonzero monomials. For example, $X^2Y + Z$ has degree three. Degrees add under multiplication, unless cancellation lowers them.

The paper's overview first assumes every clause has width at most three and every variable appears in at most three clauses, counting either sign. The total number of literal occurrences is then at most $3n$. Because a clause polynomial has degree at most its width, the whole product has degree at most $3n$. The proof will use that *linear* degree bound to keep interpolation costs under control.

The three-occurrence condition is a temporary restriction. It must be removed before ETH is refuted. In particular, an algorithm for an easy subclass of formulas would not contradict ETH's worst-case claim for arbitrary 3-SAT.

AHA 4 · A COUNT OF EXPONENTIAL MAGNITUDE NEEDS ONLY LINEAR OUTPUT LENGTH

There can be 2^n solutions, but the answer is one integer between zero and 2^n. In binary it uses at most $n+1$ bits. Returning the exact count does not require listing the solutions. The obstacle is computing that integer, not printing it.

Direct clause polynomial and exact sum: Section 2.2, equation (2.2), p. 8. Formal general sum: Section 3.1, p. 21. The output-length argument is elementary and proved here.

6. Deform the problem until groups can separate

If the variables split into independent groups with no clause connecting different groups, counting would be easier. Count the solutions in each group, then multiply those counts. For groups with at most $2n/r$ variables, where r is the number of groups, local exhaustive search examines at most $r2^{2n/r}$ assignments in total. General formulas do not possess this independence.

The paper instead makes an algebraic family that equals the desired count at one special parameter value. At other values, it arranges enough separation to compute useful information quickly. The auxiliary parameter is T. It is not an extra Boolean variable and is not a time coordinate.

The count becomes one value of a polynomial

For each occurrence of variable v in clause e, replace the formal input X_v by a linear combination of all variables with polynomial coefficients:

$$\ell_{e,v}(T, X) = \sum_{i=1}^{n} p_{e,v,i}(T) X_i.$$

A *linear form* is such a weighted sum with no products between the X_i. Each coefficient $p_{e,v,i}(T)$ is a polynomial in T of degree at most a parameter R. Require $\ell_{e,v}(0, X) = X_v$. Different occurrences of the same variable can have different forms away from zero.

Define the single-variable polynomial

$$H(T) = \sum_{x \in \{0,1\}^n} \prod_{e} \Phi_e((\ell_{e,v}(T, x))_{v \in I_e}).$$

At $T = 0$, every replacement restores its original input, so $H(0) = \#F$. In the bounded-occurrence setting, the degree of H is at most $3nR$. This follows by substituting degree-at-most-R coefficients into a product with total input degree at most $3n$. Summing over x cannot increase the degree.

At a nonzero parameter value, a replacement form may be negative or fractional. The transformed factors need not be truth indicators, and $H(T)$ need not count solutions to another SAT instance. Exact algebra, including cancellation, preserves the connection to $H(0)$. This is why a “clever guessing” description misses the mechanism.

Few shared summaries replace full cross-group assignments

Fix a selected positive integer $t \in \{1, \dots, Q\}$, where Q is the number of interpolation points. Partition the Boolean variables into r groups $S_1, \dots, S_r$. A *partition* consists of disjoint sets whose union is the full variable set. Each transformed clause is assigned to one group. Its inputs may use variables in that group freely. Dependence on all other groups must pass through a small set of shared integer linear measurements.

For group g, collect its bits into a vector x_g, and let $y_g = C_g x_g$ be its measurements. Here C_g is a matrix with integer entries; each row computes one weighted sum of the group’s bits. The same measurements serve every transformed clause, although different clauses use different combinations of them.

Suppose there are at most $2\tau n$ measurements in total, and every row of every C_g has sum of absolute coefficients at most D. The parameter τ controls the fraction of measurements; D controls their ranges. On Boolean inputs each measurement is an integer between $-D$ and D. There are therefore at most $(2D+1)^{2\tau n}$ possible tuples of measurement values.

A SUMMARY CAN FORGET INFORMATION WITHOUT LOSING THE REQUIRED SUM

For bits y, z, the summary $s = y + z$ has three possible values. The assignments $(0,1)$ and $(1,0)$ give the same value $s = 1$. If a factor uses these bits only through s, it cannot distinguish those assignments and does not need to. A sum over that group must still count both. The sizes of the sets producing $s = 0, 1, 2$ are $1, 2, 1$.

By contrast, the single measurement $x_1 + 2x_2 + 4x_3 + \cdots + 2^{n-1}x_n$ distinguishes all 2^n assignments. Having one row, or small matrix rank, is not enough. Bounded coefficient sums are essential for a small enumeration cost.

Fix a complete tuple $\mathbf{y} = (y_1, \ldots, y_r)$ of group measurements. All transformed factors assigned to group g then depend only on that group's remaining bits. Let $A_g(u; \mathbf{y})$ be the product of those factors when its local assignment is u, and let $B_g(\mathbf{y})$ be the sum of A_g over assignments u satisfying $C_g u = y_g$. Then

$$H(t) = \sum_{\mathbf{y}} \prod_{g=1}^{r} B_g(\mathbf{y}), \qquad B_g(\mathbf{y}) = \sum_{\substack{u \in \{0,1\}^{s_g} \\ C_g u = y_g}} A_g(u; \mathbf{y}).$$

Every full assignment produces exactly one measurement tuple. Within a fixed tuple, choosing one allowed assignment in each disjoint group produces exactly one full assignment. The product of the group sums expands, by distributivity, into the sum of products over all these choices. Summing over tuples includes each full assignment exactly once. Empty groups and unattainable measurement tuples contribute according to the usual empty-product and zero-sum conventions. ∎

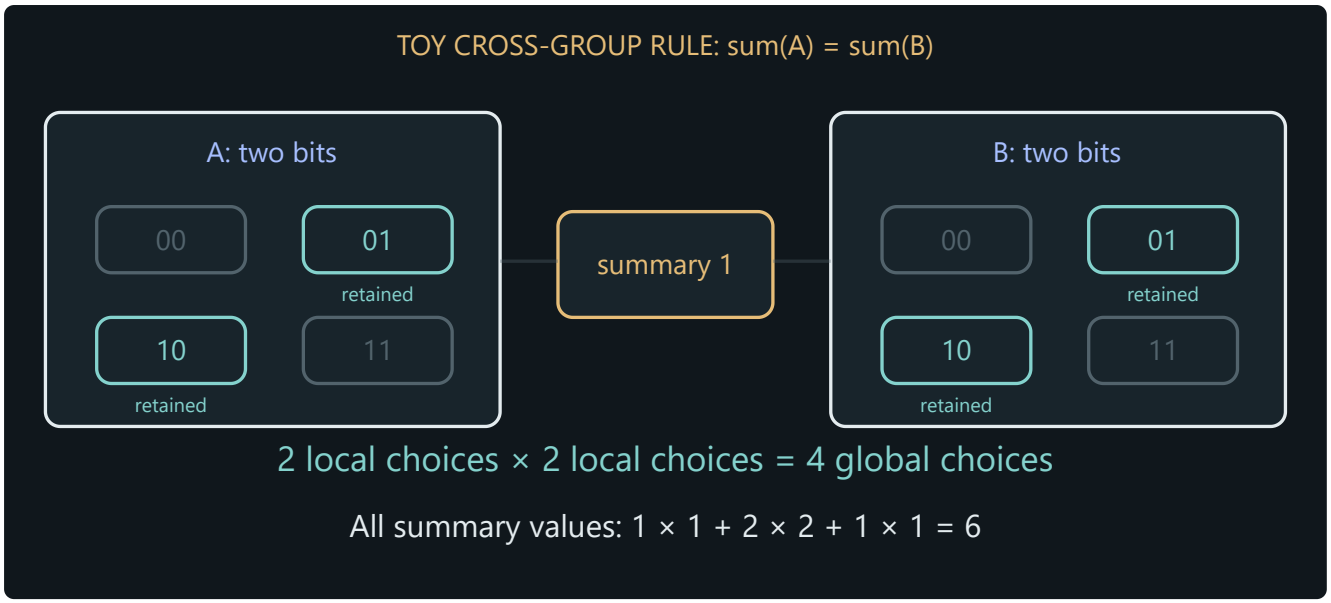
AHA 5 · SEPARATE AFTER FIXING A SMALL INTERFACE

The original formula can connect variables across every group. The algorithm does not assume those connections disappear in the input. It constructs replacements for which the dependence passes through bounded summaries at selected parameter values. The expensive work becomes "try the summaries, then scan one small group at a time."

Ignoring polynomial factors, that computation costs $(2D+1)^{2rn} 2^{2n/r}$. Both parts need accounting. Tiny groups are useless if the summaries encode the entire global assignment. Few summaries are useless if each summary has an exponential range.

Fig. 3 isolates the factorization principle in a small case. Each group contains two bits, and the rule requires their two sums to agree. Fixing the common sum leaves independent local choices. This toy already has a usable summary; the paper needs its replacement construction to obtain the corresponding structure.

A shared summary keeps multiplicities while separating choices



Fix both group sums to 1. Each group permits {01, 10}. Any retained A assignment combines with any retained B assignment. Summaries forget identities, but the local sums retain multiplicity.

Fig. 3. The rule sum(A)=sum(B) couples the groups. At fixed common sum, all retained local assignments can combine freely. Adding the three products gives 1+4+1=6. This is an exact teaching construction, not the paper's full design or deformation.

Deformation: Section 2.3, equations (2.3)–(2.4), p. 9. Shared-summary design: Definition 2.1, pp. 9–10. Separation identity: equation (2.6), p. 10; full proof in Lemma 3.2, pp. 22–23. Range warning: p. 22.

7. Recover the hard value without evaluating it directly

Computing $H(1), \ldots, H(Q)$ is not automatically enough to recover $H(0)$. A polynomial of degree d can require $d+1$ independent pieces of information. Here $d \leq 3nR$, which may be much larger than Q. The paper obtains several Taylor coefficients at each point.

Interpolation with local information

For a polynomial H and a number t, expand $H(t+z)$ in powers of a new formal variable z. Its coefficient of z^0 is $H(t)$; its coefficient of z^1 is $H'(t)$, where the prime denotes derivative. Higher coefficients are derivatives divided by the relevant factorials. No numerical approximation is involved.

Write $[z^j]H(t+z)$ for the coefficient of z^j. Let $q = \lfloor d/Q \rfloor$ and $J = q+1$, where the floor symbol means the greatest integer at most its argument. If the first J Taylor coefficients are known at each of Q distinct points, their total number QJ exceeds d. They determine H uniquely. Recovery from values and derivative information is called *Hermite interpolation*.

> If two degree-at-most-d polynomials have the same first J Taylor coefficients at t, their difference is divisible by $(T-t)^J$. If this holds at all Q distinct points, their difference is divisible by the product of those powers, whose degree is $QJ > d$. A nonzero polynomial cannot be divisible by a polynomial of higher degree. Their difference is zero. Solving the resulting linear equations therefore recovers H, in particular its constant value $H(0)$. ∎

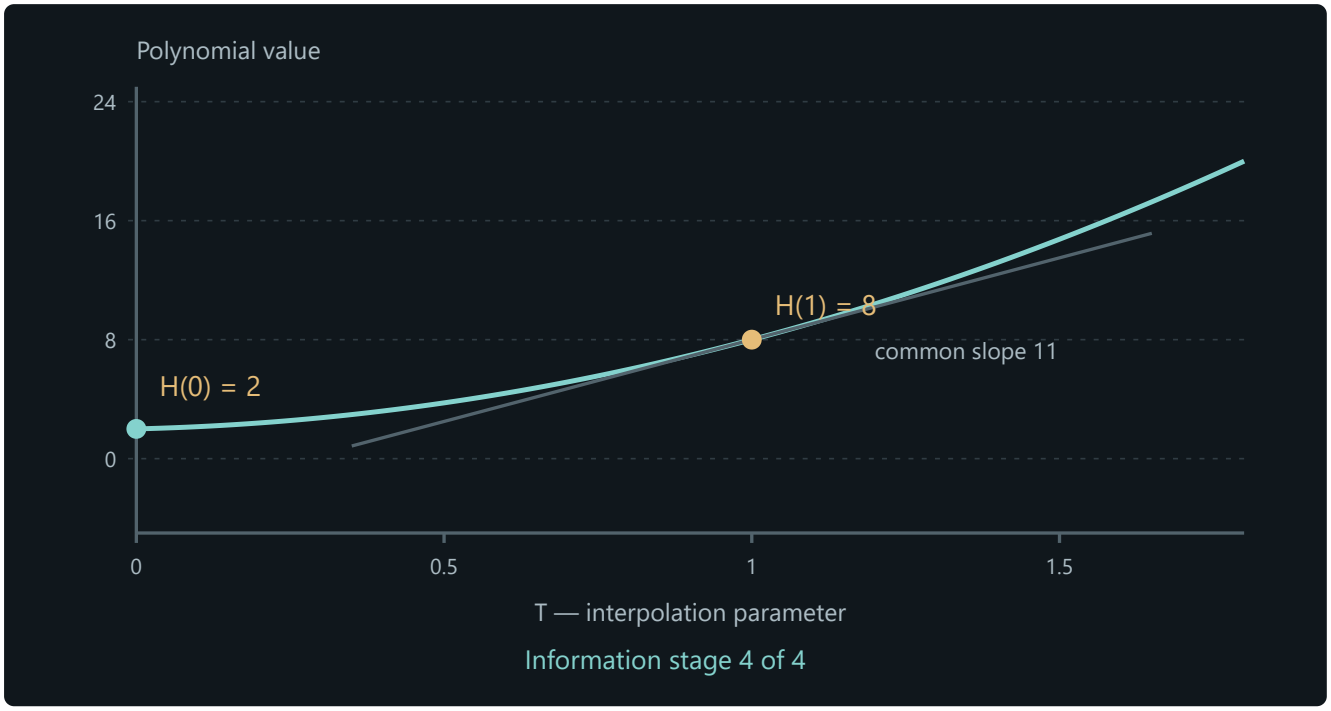
A SMALL RECONSTRUCTION

Take the teaching polynomial $H(T) = 2 + T + 5T^2$. Its value at one is eight and its derivative there is eleven. The coefficient of z^2 in $H(1+z)$ is five. Thus $H(1+z) = 8 + 11z + 5z^2$, and evaluating the recovered polynomial at $z = -1$ gives $H(0) = 8 - 11 + 5 = 2$. The value at the hard location comes from exact information at another location.

The example demonstrates reconstruction only. It gives no reason why obtaining those coefficients is cheap in a large SAT instance. That is the next obstacle.

Fig. 4 shows why the amount of local information matters. Distinct quadratics can share the same value and slope yet disagree at zero. The quadratic Taylor coefficient removes that ambiguity within the degree-at-most-two class.

Recover a value that was never directly evaluated



Now set z=−1, corresponding to T=0. Exact recovery gives H(0)=8−11+5=2. The visual shows recovery, not how the research solver obtains its data.

Fig. 4. Candidate curves are $8+11(T-1)+a(T-1)^2$ with a=1,3,5,7. All have the same value and slope at one. Specifying a=5 determines the quadratic and gives H(0)=2. This visual explains uniqueness and recovery; it does not show the research solver's method of computing its Taylor data.

Moving away from an easy point reintroduces coupling

The shared-summary property holds at $T = t$, but need not hold at $t + z$. Split each replacement into its value at t and its change:

$$\ell_{e,v}(t + z, x) = \ell_{e,v}(t, x) + \Delta\ell_{e,v}(z, x).$$

The change has zero constant term in z. In other words, every coefficient of an input bit inside this correction carries at least one factor of z. Temporarily give the corrections an independent formal input vector X, while the base forms still receive the summed Boolean assignment x. Call the resulting product $G_x(X, z)$. Setting $X = x$ restores the desired shifted product.

A term contributing to order z^j can involve at most j occurrences of correction variables, because each occurrence contributes at least one power of z. Therefore the coefficient $[z^j]G_x(X, z)$ has degree at most j in X. Retaining only orders through q restricts dependence on these corrections to low-degree polynomials.

To make that precise, write “modulo z^{q+1}” to mean discard every term with a power of z above q. This is exact truncated polynomial arithmetic, not the claim that z is a tiny real number. On Boolean inputs, every positive power of X_i equals X_i. We can therefore replace powers by first powers to obtain a *multilinear* polynomial, in which every variable’s exponent is zero or one.

Recover each small correction by inclusion–exclusion

A multilinear polynomial $p(X)$ of degree at most q is a sum of terms $a_E \prod_{i \in E} X_i$, indexed by subsets E of at most q variables. Let 1_U denote the Boolean vector that equals one on subset U and zero elsewhere. Evaluation at 1_U adds all coefficients whose index sets lie within U. Inclusion–exclusion reverses this subset sum:

$$a_E = \sum_{U \subseteq E} (-1)^{|E| - |U|} p(1_U).$$

Here $|E|$ is the number of elements in E. For a two-variable example, the coefficient of X_1X_2 is $p(1,1) - p(1,0) - p(0,1) + p(0,0)$. A constant contributes $1 - 1 - 1 + 1 = 0$; either one-variable term cancels; the two-variable term remains once. In general, an indexed coefficient cancels unless its subset equals E, by the binomial identity $(1-1)^a = 0$ for positive a.

Apply this identity to the low-order correction polynomial. In a sum over x, the factor $\prod_{i \in E} x_i$ selects assignments with $x_i = 1$ for every $i \in E$. Therefore the shifted sum can be recovered from restricted sums of the form

$$\sum_{\substack{x \in \{0,1\}^n \\ x_i = 1 \ (i \in E)}} G_x(1_U, z), \qquad U \subseteq E, \quad |E| \le q.$$

Now the correction input 1_U is fixed, so it no longer couples the bits being summed. All varying dependence on x passes through the base forms at t, where the shared-summary construction applies. The restrictions $x_i = 1$ are local to their respective groups. This recovers the Taylor coefficients using the separated counting routine.

AHA 6 · TAYLOR ORDER LIMITS THE NUMBER OF CORRECTIONS TO TRACK

The algorithm does not expand arbitrary interactions among all n variables. To recover only the first $q+1$ Taylor coefficients, it needs correction subsets of size at most q. If q is small enough relative to n, the number of such restricted problems is subexponential. Exact cancellation then reconstructs the hard count.

The decisive cost ledger

For a subset of size j, there are $\binom{n}{j}$ choices for E and 2^j choices for $U \subseteq E$. The binomial coefficient $\binom{n}{j} = n!/(j!(n-j)!)$ counts j-element subsets. Here $n! = 1 \cdot 2 \cdots n$ for positive integers n, and $0! = 1$. Hence the number of correction pairs is

$$C_q = \sum_{j=0}^{\min(q,n)} \binom{n}{j} 2^j.$$

For example, at $n = 5, q = 2$, this is $1 + 5 \cdot 2 + 10 \cdot 4 = 51$, while allowing all subset sizes gives $3^5 = 243$. The example only checks the combinatorics. The asymptotic saving requires q/n to tend to zero with a controlled bound.

Up to polynomial factors, the paper's overview has three multiplying costs:

$$\underbrace{C_q}_{\text{Taylor corrections}} \quad \underbrace{(2D+1)^{2\tau n}}_{\text{measurement tuples}} \quad \underbrace{2^{2n/r}}_{\text{one group}}.$$

Since $q = O(nR/Q)$, the elementary binomial estimate $\binom{n}{j} \le (en/j)^j$ gives $\log C_q = O(n(R/Q)\log(2Q/R))$ in the intended regime. Here e is the base of the natural exponential. Taking logarithms turns the product of costs into the sum

$$O\left(n\left[\frac{R}{Q}\log\frac{2Q}{R} + \tau\log(2D+1) + \frac{1}{r}\right]\right).$$

The construction makes all three bracketed quantities tend to zero together. In the overview, it achieves the orders

$$r = \Theta\left(\frac{\sqrt{\log n}}{(\log\log n)^2}\right), \quad Q = \Theta\left(\frac{\log n}{\log\log n}\right), \quad R = \Theta(\sqrt{\log n}), \quad \tau = \Theta\left(\frac{1}{r\log n}\right), \quad D = r^{2Q} \le (2n)^{1/128}.$$

The first bracketed term is $O((\log\log n)^2/\sqrt{\log n})$. The measurement term is $O(1/r)$ because $\log D = O(\log n)$, and the last term is also $O((\log\log n)^2/\sqrt{\log n})$. Thus the bounded-occurrence algorithm has time

$$n^{O(1)} 2^{O(n(\log\log n)^2/\sqrt{\log n})}.$$

The limit already proved says this exponent is $o(n)$. This is where the runtime argument escapes every fixed positive exponential rate. It is not merely an improvement from base two to base 1.3.

> The parameters above describe a sufficiently large-n proof regime, not a recommended implementation. The manuscript checks feasibility and uses exhaustive search on the finitely many exceptional sizes. Its asymptotic constants absorb that fallback. This prevents a formal gap at small sizes but gives no useful practical threshold.

Taylor recovery: Sections 2.5–2.6, pp. 11–13. Inclusion–exclusion identity: Lemma 3.3, p. 24. Full counting routine and costs: Proposition 3.4, pp. 25–26. Explicit parameter checks: Lemma 4.8, pp. 34–35.

8. Why the required low-degree replacements can exist

The cost calculation is conditional on obtaining an unusual combination of properties: many useful evaluation points, low-degree replacements, small groups, and few bounded measurements. Choosing an unrelated easy replacement at each of Q points would usually require degree about Q. That would make R/Q large and ruin the Taylor saving. Section 4 is the structural part of the paper that addresses this obstruction.

Split a coordinate into a measured part and an almost-local part

The construction first operates on an auxiliary vector ζ of length D. These coordinates record possible group schedules across the Q points. They are placeholders that will later contain original input variables or zero. They are not D new independently chosen Boolean variables.

Let I denote the identity matrix, which leaves every vector unchanged. Seek a square matrix M_0 such that $I - M_0$ can be determined from a short measurement vector $C\zeta$. Then, for auxiliary coordinate u,

$$\zeta_u = ((I - M_0)\zeta)_u + \sum_w M_0[u, w]\zeta_w.$$

The *kernel* of a matrix C is the set of vectors it sends to zero. If $M_0\xi = \xi$ for every ξ in that kernel, then $(I - M_0)\zeta$ is determined by $C\zeta$: any two vectors with the same measurements differ by a kernel vector, on which $I - M_0$ vanishes. This proves the needed information property without requiring the measurements to determine the entire vector.

For each nonzero coefficient $M_0[u, w]$, let $E_{u,w}$ be the set of evaluation points where coordinate w belongs outside coordinate u's assigned group. Require at most $R - 1$ such mismatches. Multiply that coefficient by

$$\varphi_{u,w}(T) = \prod_{s \in E_{u,w}} (1 - T/s).$$

This multiplier is one at zero and zero at every offending point $T = s$. Its degree is the number of offending points, at most $R - 1$. Leave the measured term unchanged. The resulting replacement equals the original coordinate at zero, and at each selected nonzero point uses only its assigned group plus measurements. The challenge has become constructing M_0 with a small exceptional measurement space and few mismatches per nonzero entry.

One interpolation polynomial solves two constraints

The auxiliary coordinates are indexed by Q pairs of group labels. Swapping the two labels in position s defines a permutation matrix Π_s. The swaps commute, meaning their order does not matter, and each squares to the identity. Define

$$V_0 = \prod_{s=1}^{Q} \Pi_s, \qquad W_0 = \frac{1}{2} \sum_{s=1}^{Q} (I - \Pi_s), \qquad M_0 = V_0 f(W_0),$$

where f is an ordinary polynomial to be chosen. The notation $f(W_0)$ means substitute the matrix into the polynomial, using matrix multiplication for powers. A vector $\psi \neq 0$ is an *eigenvector* of a matrix if applying the matrix multiplies it by a scalar, its eigenvalue.

For one position, let e_{ab} be the coordinate vector with a one at group-label pair (a, b) and zeros elsewhere. The swap fixes e_{aa} and $e_{ab} + e_{ba}$, and negates $e_{ab} - e_{ba}$ for $a < b$. These vectors form an *orthogonal basis*: the dot product, obtained by multiplying corresponding entries and adding, is zero between distinct basis vectors. There are r diagonal vectors and $r(r-1)/2$ unordered pairs with $a < b$. Thus there are $r(r+1)/2$ positive-eigenvalue vectors and $r(r-1)/2$ negative-eigenvalue vectors, for a total of r^2. A basis is a set of linearly independent vectors from which every vector can be formed as a linear combination.

Choose one such basis vector at each of the Q positions and multiply their coordinate entries to construct a vector on the full auxiliary array. Inner products of these product vectors factor into inner products at individual positions, so the product vectors also form an orthogonal basis. They are eigenvectors of every swap simultaneously. If a product vector ψ has j negative swap eigenvalues, then

$$W_0\psi = j\psi, \qquad V_0\psi = (-1)^j\psi, \qquad M_0\psi = (-1)^j f(j)\psi.$$

Thus M_0 fixes that vector whenever $f(j) = (-1)^j$. Among the r^2 choices at one position, a fraction $(r-1)/(2r)$ have negative eigenvalue. Counting all product-basis choices therefore gives a binomial distribution for j, the number of successes in Q independent two-outcome trials. This counts basis vectors; the final SAT algorithm is deterministic. Most j values lie near their mean within a width on the order of $\sqrt{Q}$.

Choose an interval containing R consecutive integers around that mean, and interpolate $f(j) = (-1)^j$ on it. Degree at most $R-1$ suffices. Taking R much larger than $\sqrt{Q}$ but much smaller than Q covers almost all the basis vectors while retaining low degree. Measure the dot product with each exceptional basis vector outside the interval. Because the basis is orthogonal, a vector with all those measurements zero is a combination only of the retained directions, all fixed by M_0. Hence M_0 fixes the measurement kernel, exactly as required.

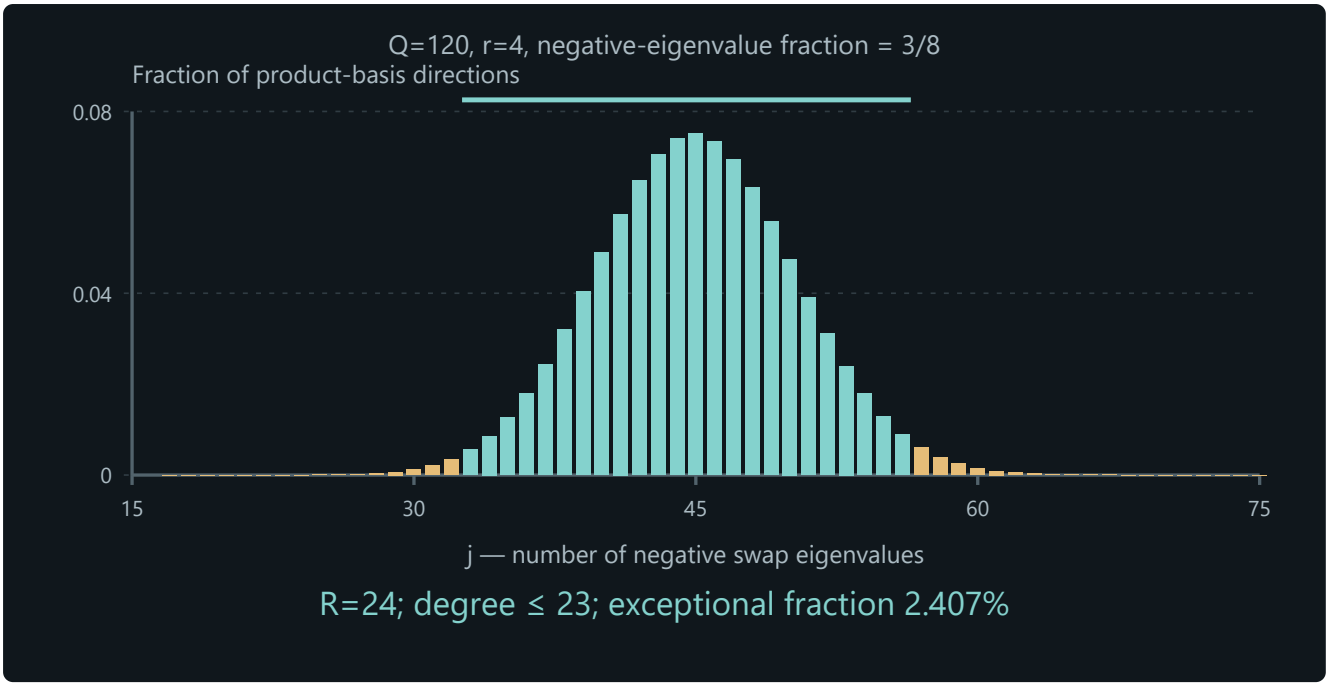
At the same time, degree at most $R-1$ means expanding $f(W_0)$ uses products of at most $R-1$ swaps. Multiplication by the complete swap V_0 then permits at most $R-1$ positions where the complete-swap matching is undone. That proves the mismatch bound for nonzero entries. The same polynomial therefore controls both the number of exceptions and how nonlocal each surviving term can be.

AHA 7 · LOW DEGREE AND CONCENTRATION FIT IN THE SAME WINDOW

The construction uses an interval wide enough to cover almost all spectral directions, yet narrow enough for low-degree interpolation. The regime $\sqrt{Q} \ll R \ll Q$ makes those demands compatible. Exceptional directions are measured; the rest are localized by polynomial zeros. This is a more specific explanation than saying the algorithm “detects hidden patterns.”

Fig. 5 makes the degree-versus-exceptions tradeoff visible. It counts product-basis directions by their number of negative swap eigenvalues. The binomial distribution here concerns basis directions, not random SAT assignments.

Cover most directions with a short interpolation interval



Retain j=33 through 56 using a degree-at-most-23 interpolation polynomial. Teal directions are fixed; gold directions require measurements. Increasing R reduces exceptions but increases interpolation degree. Tails outside the displayed range are included in the exact fraction.

Fig. 5. Exact finite illustration with Q=120 and r=4, giving negative-eigenvalue fraction 3/8. Teal bars lie inside the retained R-point interval; gold bars are exceptional. Displayed fractions include tails outside the plotted range. These choices illustrate the mechanism; they do not establish the full asymptotic feasibility conditions of Lemma 4.8.

An exact four-coordinate demonstration

Remark 4.4 gives a small example. Take four auxiliary values $\zeta_0, \zeta_1, \zeta_2, \zeta_{12}$, define the single measurement $c = \zeta_0 - \zeta_1 - \zeta_2 + \zeta_{12}$, and use

$$P(T) = \zeta_0 - \frac{T}{2}\zeta_1 - T\zeta_2.$$

Then

$$P(0) = \zeta_0, \qquad P(1) = c + \tfrac{1}{2}\zeta_1 - \zeta_{12}, \qquad P(2) = c - \zeta_2 - \zeta_{12}.$$

Substituting the expression for c verifies both identities. At point one the assigned group contains coordinates ζ_1, ζ_{12}; at point two it contains ζ_2, ζ_{12}. Once c is fixed, each evaluation is local to its assigned group. At zero the original coordinate is restored exactly.

The example has $Q = R = 2$, outside the asymptotic requirement $R < Q$. It demonstrates the mechanism, not the runtime theorem. The full construction supplies this property for every needed occurrence, balances the group sizes, bounds coefficient sizes, and keeps the total number of measurements small. Its auxiliary copies repeat the same original variable values, rather than introducing independent bits that would change the count.

Construction overview: Section 2.7, pp. 13–18. Full matrix and support arguments: Section 4.2, Lemmas 4.1–4.2, pp. 27–29. Polynomial rows: Lemma 4.3, p. 29. Four-coordinate example: Remark 4.4, p. 30. Schedule balancing and transfer: Sections 4.4–4.6, pp. 30–34.

9. Remove the special-input restrictions

A fast algorithm for three-occurrence formulas is not yet an algorithm for arbitrary 3-SAT. The paper gives a relatively direct route to general fixed-width formulas, sufficient for the qualitative ETH refutation, and a more efficient route that avoids excessive variable growth. Understanding the first route separates the logical refutation from the sharper exponent.

Route A: a weaker bound already refutes ETH

The *sparsification lemma* decomposes a fixed-width formula into a controlled collection of formulas with fewer occurrences per variable. A counting version needs more than "the original is satisfiable if one leaf is." It must partition satisfying assignments so that no assignment is counted in two leaves.

For a rational parameter σ with $0 < \sigma \leq 1$, the paper's Lemma 6.3 produces at most $\mathrm{poly}_k(n, \sigma^{-1})2^{\sigma n}$ nodes in a branching tree. Each leaf uses the same declared variables, and its variables have occurrence bound

$$B = O_k\big((\sigma^{-1}\log(2/\sigma))^{k-1}\big).$$

The counts add because the leaf solution sets are disjoint. A small disjointness example shows the issue. The branches $x = 1$ and $y = 1$ both include $(x, y) = (1, 1)$, so adding their counts overcounts the formula $x \vee y$. Branching instead into $x = 1$ and $(x = 0, y = 1)$ partitions its three solutions. Appendix A proves the quantitative branching lemma; it is not an assumption that every sparse decomposition preserves counts.

Each sparse leaf can be converted into a width-at-most-three formula by introducing uniquely determined gate variables for long clauses. A high-occurrence variable can then be replaced by one copy per occurrence, with a directed cycle of implications forcing all copies to agree. Each copy appears in its original clause and two cycle clauses. This gives at most three occurrences per variable and preserves counts bijectively.

The number N of variables after this conversion is $O_k(Bn)$. Applying the bounded-occurrence algorithm therefore pays for this blowup. Choosing σ within a constant factor of $(\log n)^{-1/(2k)}$, as in Section 2.8, gives the weaker bound

$$(n+m)^{O_k(1)}2^{O_k\left(n(\log\log n)^{k+1}/(\log n)^{1/(2k)}\right)}.$$

For each fixed k, the exponent divided by n tends to zero: any fixed power of $\log\log n$ grows more slowly than the positive power $(\log n)^{1/(2k)}$. This is the same exponential-versus-power limit used earlier. Set $k=3$. Testing whether the count is positive then decides arbitrary 3-SAT in subexponential time. **This is already the claimed qualitative contradiction to ETH.**

> **AHA 8 · YOU CAN LOCATE THE REFUTATION BEFORE THE SHARPEST THEOREM**
>
> Read Sections 2.2–2.7 to see the special-case mechanism, then Section 2.8 to remove its restrictions. The resulting exponent is weaker than Theorem 1.1's, but still sublinear. Appendix D improves the bound; its improvement is not needed for the bare conclusion that ETH is false.

Route B: compress clause checks without adding variables

To avoid the variable-copying penalty, the main proof keeps the original n Boolean variables. It replaces many clauses by $O(n)$ sparse polynomial equations whose common solutions are exactly the satisfying assignments. A *common zero* is an assignment making every polynomial in a system equal to zero.

Work first over $\mathbb{F}_2$, the two-element field. Addition is modulo two, so $1+1=0$, and multiplication is ordinary bit multiplication. Let $v_e(x)$ be the violation bit for clause e: one if violated, zero otherwise. A *parity test* on a set of clauses requires the sum of their violation bits to be zero modulo two. It accepts an even number of violations.

For the clause $x \vee \neg y \vee z$, the violation polynomial is $(1+x)y(1+z)$ over $\mathbb{F}_2$. It equals one exactly when $x=0, y=1, z=0$, the unique violating pattern on these variables. A parity test sums a short list of such polynomials. Every satisfying assignment passes every test. A nonsatisfying assignment may also pass, so this alone is not an exact reduction.

Use the counter to construct a perfect filter

Initial parity checks come from an error-correcting code. A binary linear code is a set of bit strings satisfying specified parity equations. Its minimum-distance property requires every nonzero accepted string to have many nonzero positions. Applied to the vector of clause violations, this means any nonsatisfying assignment surviving the initial tests must violate many clauses. Lemma 6.1 constructs

suitable short checks using an expanding graph, whose edges prevent a small set of violations from remaining isolated.

Once a surviving bad assignment has many violations, a short randomly chosen test rejects it with probability at least $1/4$. Therefore, averaged over candidate tests, at most $3/4$ of the currently surviving bad assignments remain. Some candidate achieves that average or better.

The algorithm chooses this candidate deterministically. For each candidate, it counts the solutions of the resulting sparse polynomial system, using a counting subroutine already established in Section 5 of the paper. This subroutine works on arbitrary systems of the specified size and sparsity, even when they still admit bad assignments. It is not a call to the final general #SAT algorithm being constructed.

The candidate family also needs a cost bound. At one sparse leaf, let m_G be its number of clauses and let P_{test} be the test-list length. Each position chooses a clause index and an inclusion bit, giving $(2m_G)^{P_{\text{test}}}$ lists. Repeated clause indices are combined modulo two. The paper constructs P_{test} bounded by a fixed power of $\log n$, and $m_G \le n(\log n)^{O_k(1)}$. The candidate count is therefore $\exp((\log n)^{O_k(1)})$. Over $3n$ rounds this adds only a fixed power of $\log n$ to the time exponent. Such a power is smaller than $n(\log\log n)^2/\sqrt{\log n}$ for sufficiently large n, so enumeration is absorbed in the stated bound. Section 6.3 of the paper also includes the construction cost of the initial code checks.

If the count is $N = \#F + D_{\text{bad}}$, where D_{bad} is the number of surviving nonsatisfying assignments, then minimizing N also minimizes D_{bad}: the unknown $\#F$ is unchanged across candidates. Append a minimizing test and repeat for $3n$ rounds. The number of bad assignments left is at most

$$2^n(3/4)^{3n} = (27/32)^n < 1.$$

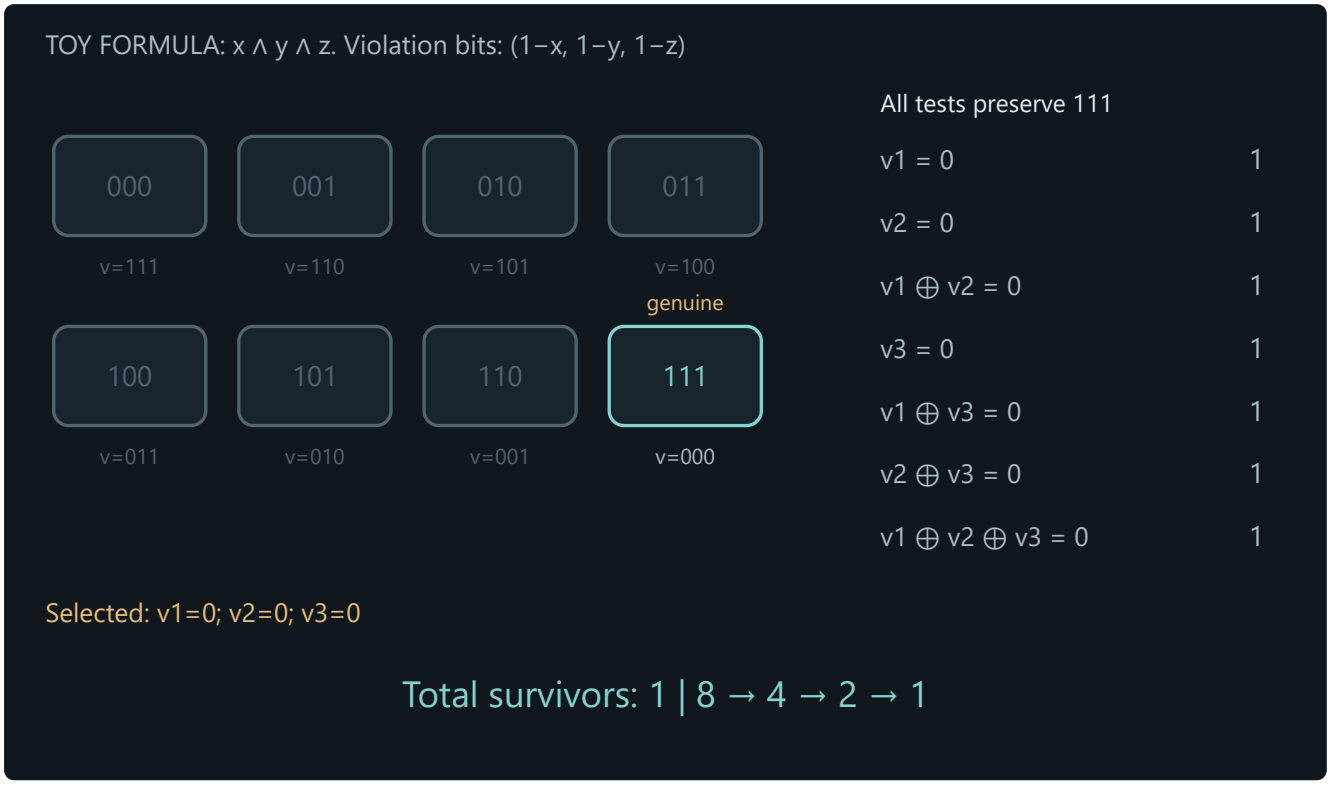
That number is a nonnegative integer, so it must be zero. The final system has exactly the desired solutions. The construction does not need to know $\#F$ or to detect the moment the bad count becomes zero.

AHA 9 · COUNTING BUILDS THE REDUCTION THAT COUNTING THEN USES

A separately proved counter for sparse polynomial systems lets the algorithm choose parity tests by their exact remaining counts. Because genuine solutions survive every candidate, comparing total counts identifies the best rejection of false positives without knowing the genuine count. This is an algorithmic use of averaging, not a circular appeal to a #SAT oracle.

Fig. 6 shows the candidate-comparison logic on the toy formula $x \wedge y \wedge z$. Its violation bits are $v_1 = 1 - x$, $v_2 = 1 - y$, and $v_3 = 1 - z$. The symbol $\oplus$ denotes addition modulo two. A test can accept a false positive when its selected violation bits cancel. Each round chooses a test using only the total surviving count.

Choose a filter without knowing the genuine solution count



Three greedy choices leave only 111, the genuine solution. Candidate ranking used only total surviving counts. This finite example enumerates every test; the paper needs its separate sparse-test construction and cost bounds.

Fig. 6. All seven nonempty parity tests are enumerated on this finite example. Greedy choices leave 8, 4, 2, then 1 assignments. The only genuine solution, 111, survives throughout. This checks the comparison principle, not the paper's expander-code construction, sparse-test generator, or general runtime bound.

A modular trick prevents a quadratic degree explosion

The sparse-system counter also needs an algebraic representation of low total degree. Lift each parity equation to an ordinary integer polynomial $L_i(X)$ by summing its listed monomials over the integers. The equation over $\mathbb{F}_2$ holds exactly when this integer value is even. Let $\ell = n + 1$, so that $2^\ell > 2^n$. Knowing the total solution count modulo 2^ℓ determines it exactly.

Congruence $a \equiv b \pmod{2^\ell}$ means that $a - b$ is divisible by 2^ℓ. Lemma 5.1 constructs an integer polynomial $p_\ell(Y)$ that is congruent to one on even integers and zero on odd integers. Multiplying one indicator per test would normally give total input degree on the order of n^2. That would destroy the low-degree recovery budget.

The useful representation is

$$p_\ell(Y) = \sum_{j=0}^{\ell-1} e_j(Y) h(Y)^j, \qquad h(Y) = Y(Y-1), \qquad \deg e_j \le 1.$$

Every integer value of h is even, since one of two consecutive integers is even. A product containing at least ℓ factors of h, from any combination of tests, is therefore divisible by 2^ℓ and contributes zero to the required residue. Discard those terms *across the full product*. Retained terms have at most one degree-one factor per test and at most $\ell - 1$ degree-two factors in total.

If there are $a = O(n)$ tests and each L_i has degree at most fixed k, the retained input degree is at most $k(a + 2(\ell - 1)) = O(n)$. A formal marker records the number of h factors during multiplication, so the algorithm does not expand an uncontrolled multivariate polynomial. This restores the linear degree budget needed by the deformation routine.

THE MOD-FOUR CASE EXPOSES THE MECHANISM

For $\ell = 2$, the paper gives $p_2(Y) = 1 - 3Y^2 + 2Y^3 = (1 - Y) + (2Y - 1)h(Y)$. At $Y = 0, 1, 2, 3$, its values are $1, 0, 5, 28$, with residues $1, 0, 1, 0$ modulo four.

In a product $p_2(y_1)p_2(y_2)$, the term containing $h(y_1)h(y_2)$ is divisible by four. Removing it leaves the same residue and reduces the product degree from six to at most four. The full proof uses the same divisibility argument at precision 2^{n+1}.

Sections 3–6 combine this system counter with disjoint sparsification and deterministic test selection. Proposition 6.4 obtains the bound with exponent $O_k(n(\log\log n)^2/\sqrt{\log n})$ on arbitrary fixed-width formulas, using polynomial space. Appendix B checks bit lengths so that exact rational arithmetic does not conceal exponential work in one arithmetic operation.

Qualitative route: Section 2.8, pp. 18–19. Efficient route overview: Section 2.9, pp. 19–21. Parity lift and truncation: Lemmas 5.1–5.2, pp. 35–37. Counter on arbitrary sparse systems: Theorem 2.2 proof, pp. 38–39. Test selection: Lemma 6.2, p. 42. General count: Proposition 6.4, pp. 43–44. Disjointness and arithmetic: Appendices A–B, pp. 44–51.

10. Appendix D sharpens the exponent through recursion

The revised Theorem 1.1 states a better general exponent with one factor of $\log \log n$, rather than its square. It also states a fixed-density bound without that numerator. These come from Appendix D, a different organization of the algebraic counting machinery using Boolean tensor networks.

> **A TENSOR NETWORK, FROM ITS SMALLEST PIECES**
>
> A *tensor* here is a table: for each tuple of bits on its incident connections, it returns a number or a specified algebraic element. The connections are called wires or edges. A *contraction* sums, over all wire-bit assignments, the product of all table entries. Neighboring tables share the bit on their connecting wire.

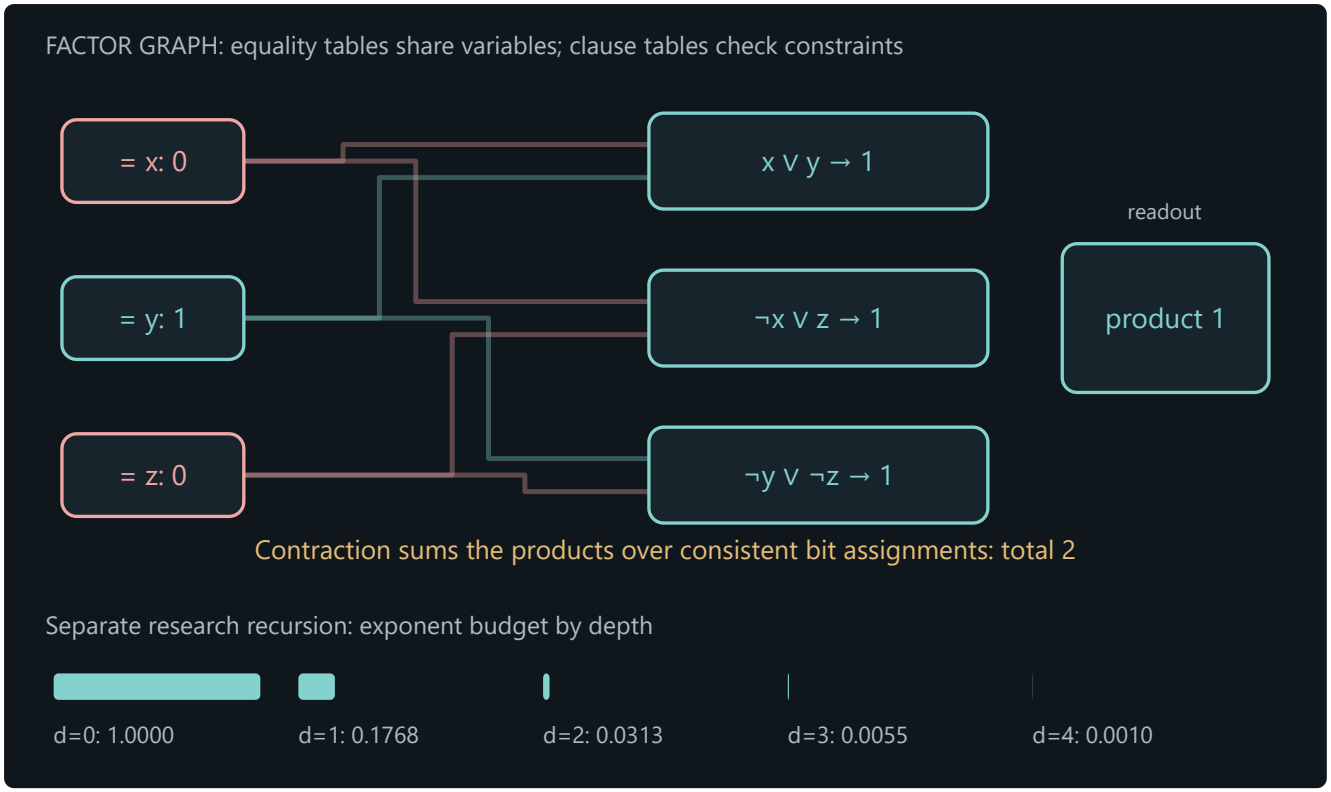
A clause is a table returning one when its incident bits satisfy it and zero otherwise. An equality table returns one when copies of a variable agree and zero otherwise. Connecting those tables makes the contraction count consistent satisfying assignments. An equality tree ensures all occurrences of a variable agree using small-degree tables and uniquely determined internal wires. Unused declared variables contribute a separate factor of two.

If a formula has $m \leq D_0 n$ clauses for a fixed density constant D_0, this encoding uses $O_{k,D_0}(n)$ wires. The theorem's notation uses D for this fixed density constant; D_0 here avoids confusion with the measurement-range parameter from the earlier construction. Appendix D's contraction theorem, Theorem D.1, gives time $\mathrm{poly}(s, L)2^{O(s/\sqrt{\log s})}$ for networks with $O(s)$ wires under its entry-computation, degree, and arithmetic hypotheses. Here L is the network-description bit length, and the clause encoding satisfies those hypotheses for fixed width and sufficiently large size.

Putting $s = O(n)$ gives the claimed fixed-density count in time $\mathrm{poly}_{k,D_0}(n, m)2^{O_{k,D_0}(n/\sqrt{\log n})}$. For arbitrary density, the sparse polynomial-system route instead yields a network representation with size parameter $s = O_k(n \log \log n)$. Applying the same contraction bound gives the general exponent $O_k(n \log \log n/\sqrt{\log n})$.

Fig. 7 displays the running formula as a factor graph: each equality table enforces one shared variable value, and each clause table checks its incident bits. Its lower panel separately previews the decay of the research proof's recursion-cost bound. The two panels describe different scales; the small network is not an execution trace of Appendix D.

From a formula to a network contraction



Assignment 010 contributes 1. Equality tables require all ports of a variable to carry the same bit. The research proof contracts much larger networks recursively: each depth's exponent is multiplied by $\sqrt{2}/8 \approx 0.1768$. The bars illustrate that bound, not a run of the research algorithm.

Fig. 7. Equality tables tie together the separate edge bits for each variable; the contraction sums the resulting products and equals two. The bars show the separate research recurrence's normalized exponent terms $(\sqrt{2}/8)^d$, not measured running times. Crossings without junctions do not connect edges.

What makes the contraction theorem work?

The contraction proof deforms wire identifications so that at chosen interpolation nodes only ports in the same group interact. Near each node, the remaining crossing interactions can be expanded in auxiliary variables called *modes*. The weight of a mode records a guaranteed power of two dividing its coefficient. This is an algebraic expansion, not a physical vibration or a neural-network activation.

For precision modulo 2^p, any product of modes with total weight at least p vanishes. Only lower-weight monomials need to be enumerated. A graded counting estimate bounds their number. The precision itself

must also be controlled, because interpolation can introduce denominators with powers of two.

The *two-adic valuation* $v_2(a)$ of a nonzero integer a is the exponent of the largest power of two dividing it. For example, $v_2(12) = 2$ and $v_2(40) = 3$. It extends to rational numbers by subtracting denominator valuation. The proof assigns different numbers of Taylor coefficients to interpolation nodes with different valuations. This tapering limits precision loss and hence the number of modes that survive.

Lemma D.5 proves the tapered recovery step. Equations (D.14)–(D.16) bound the surviving modes and factor their contributions into smaller child contractions. Each child has fewer than one eighth as many wires as its parent. The children are processed sequentially so that an enormous search tree need not be stored.

WHY THE RECURSION HAS A SUBLINEAR EXPONENT

At recursion depth d, let M be the number of wires and u the number of retained valuation levels. The proof gives $M = O(s8^{-d})$, $u = \Theta((\log s)/2^d)$, and an expansion cost $\exp(O(M/\sqrt{u}))$. Here $\exp(a) = e^a$. Thus the cost exponent at that depth is bounded by a constant times

$$\frac{s}{\sqrt{\log s}}\left(\frac{\sqrt{2}}{8}\right)^d.$$

The ratio $\sqrt{2}/8 \approx 0.176777$ is less than one. Summing these exponents over the recursion path gives a convergent geometric sum with multiplier $1/(1-\sqrt{2}/8) \approx 1.214737$ before hidden constants. Polynomial branching factors and base-case work are included in the paper's full estimate. The final exponent remains $O(s/\sqrt{\log s})$.

AHA 10 · THE INTERACTIONS VANISH BY DIVISIBILITY, THEN THE REMAINING PROBLEM SHRINKS

Two mechanisms cooperate: high-weight crossing terms contribute zero at the required modular precision, and each retained term breaks into substantially smaller contractions. Tapered interpolation prevents precision costs from cancelling that saving. This is the turning point behind the sharpest bound, beyond the simpler ETH-refuting route in Section 2.8.

> This section is a dependency guide to Theorem D.1, not a complete proof of its packing, valuation, and arithmetic lemmas. Its exact ring construction and interpolation estimates are substantial. The relevant paper steps are linked below so that the summary never substitutes for the assertions on which the theorem depends.

Definition and theorem: Appendix D.1, Theorem D.1, p. 53. Fixed density: D.2, p. 54. Arbitrary density and network encoding: D.3, including Lemma D.2, pp. 54–57. Packing and deformation: D.4.2–D.4.3, pp. 58–60. Tapered recovery: Lemma D.5, pp. 60–61. Modes and child networks: D.4.5–D.4.6, pp. 62–63. Recurrence and space: D.4.7, p. 64.

11. What has been claimed, checked, and left open

The exact logical chain

1. **A general counting guarantee.** The manuscript's Theorem 1.1 states that every fixed-width k-CNF can be counted deterministically with polynomial input overhead and a sublinear exponent in n.

2. **Choose the required width.** Set $k = 3$, rather than letting k grow with the input.

3. **Turn counting into decision.** Answer yes if and only if the exact count is positive.

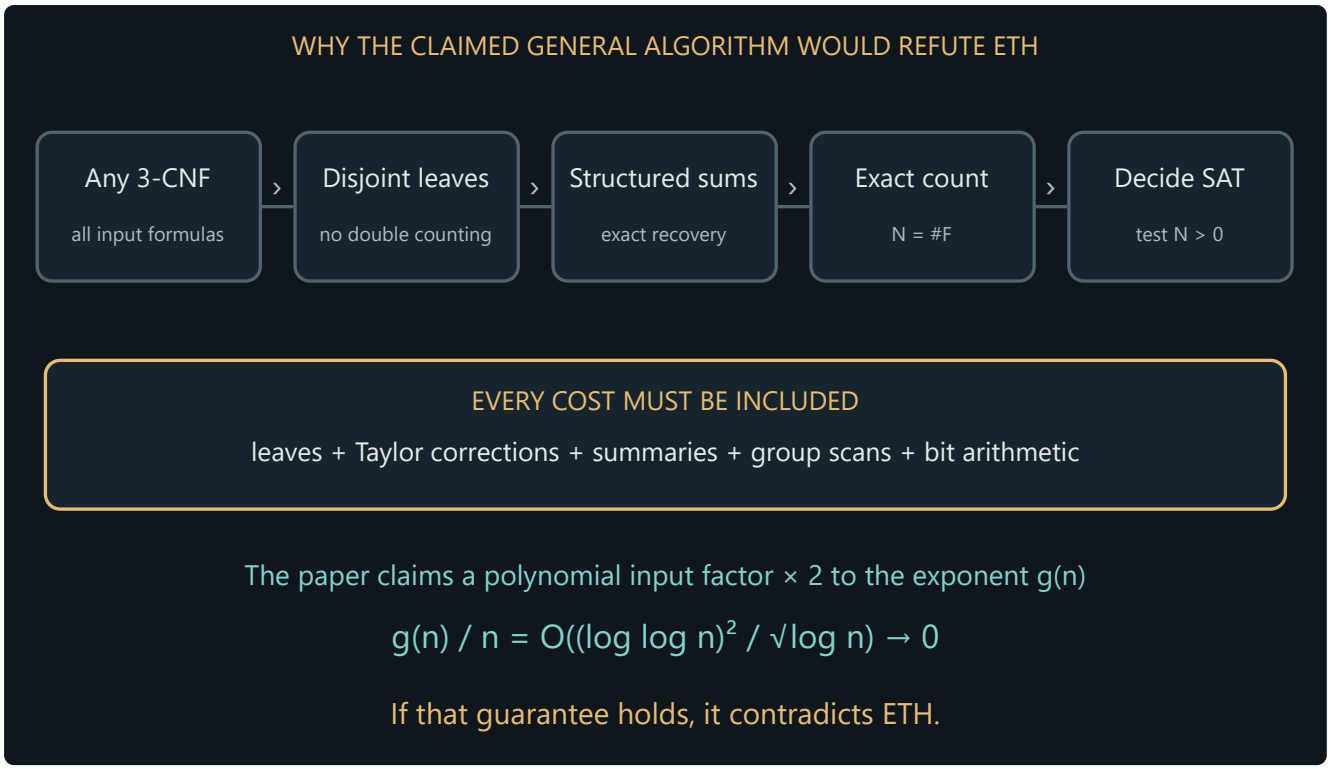
4. **Compare growth rates.** By the limit calculation, the claimed exponent is $o(n)$. Eventually it is smaller than δn for any fixed positive δ.

5. **Contradict the hypothesis.** This is the deterministic subexponential 3-SAT algorithm that ETH rules out, if the stated research guarantee is correct.

The logical deduction from the runtime guarantee to the refutation is short. Establishing the guarantee is the difficult mathematical work. A clear distinction between the two keeps the excitement focused on the actual achievement being claimed.

Fig. 8 connects the earlier mechanisms to that deduction. It marks the two obligations that the finite examples cannot establish: correctness on every input and the complete resource bound. The final implication is conditional on those research results.

Why the claimed algorithm would make ETH false



The decisive point is total cost: g(n)/n tends to zero. For every fixed positive exponential rate δ, the normalized exponent is eventually smaller. A deterministic general 3-SAT algorithm with this bound is what ETH forbids. The research guarantee remains attributed, not independently certified here.

Fig. 8. This is a visual argument map, not a new proof or execution of the research algorithm. The paper supplies the general construction and runtime accounting; the tutorial proves the elementary positivity test and sublinear-exponent limit. See the three-cost ledger, generality, and the precise bounds. The weaker squared-log-log bound displayed here already suffices to refute ETH if correct.

The three bounds must not be merged

Let $\lambda(n) = \log n$ and $\nu(n) = \log\log n$ for this table only. Each time bound also has the polynomial input-size factor described in its source. Constants may depend on the fixed parameters.

Source and scope	Exponential factor	Role
Sections 3–6; repository Lean scope note for canonical fixed-width inputs	$2^{O_k(n\nu(n)^2/\sqrt{\lambda(n)})}$	Already subexponential. The scope note uses encoding length L for its polynomial factor.
Theorem 1.1, arbitrary clause density; Appendix D	$2^{O_k(n\nu(n)/\sqrt{\lambda(n)})}$	Sharper manuscript guarantee. The reviewed scope note does not claim this sharper

Source and scope	Exponential factor	Role
		exponent as its formalized bound.
Theorem 1.1, $m \leq D_0 n$ for fixed D_0; Appendix D.2	$2^{O_{k,D_0}(n/\sqrt{\lambda(n)})}$	Stronger guarantee under fixed clause density.

What does a Lean proof establish?

A *proof assistant* represents mathematical definitions, theorem statements, and proof objects in a formal language. A small checking kernel verifies that a proof follows from the specified rules and assumptions. Lean is one such system. It can check an argument about an algorithm without running that algorithm on large instances.

Three questions remain distinct: does the formal proof check; does its formal statement express the intended mathematical claim; and does that claim imply the advertised practical consequences? A successful kernel check addresses the first question within its trusted foundations. Reading the definitions, encodings, resource model, and correspondence addresses the second. Benchmarking and engineering would be needed for practical consequences.

The repository's scope document describes an actual deterministic machine, exact count, time bound, and polynomial bound on visited tape cells. The associated comparator statement spells out these requirements. Its challenge file ends with `sorry`, an admitted placeholder in that *target statement*; the comparator configuration points to a separate solution module, `OAI.Algorithms.SubexponentialSAT.Main`, which imports the formalization. The placeholder in the challenge is therefore not, by itself, evidence that the separate proof is admitted. Nor is reading the challenge a verification of that proof.

No Lean compilation, comparator run, complete dependency/axiom audit, or executable SAT implementation was performed for this tutorial. The elementary examples and teaching derivations were checked separately. The exact research theorem is attributed to the supplied paper and the repository's stated scope.

Formalization sources: scope document; comparator target statement; configuration and permitted axioms; separate solution entry point. Snapshot inspected 27 October 2026; file hashes are recorded in the source manifest.

Which consequences follow, and which need new arguments?

P versus NP remains undecided by the claimed bound. A subexponential upper bound neither supplies a polynomial algorithm nor establishes a superpolynomial lower bound. The inference “ETH false, therefore P = NP” is invalid. So is “the displayed bound is superpolynomial, therefore P ≠ NP.”

Conditional hardness results need their assumptions re-examined. If a lower bound relied only on ETH or SETH, that premise would no longer establish it. The underlying reduction can remain valid. Some conclusions may have alternative support. The paper specifically notes that refuting SETH does not by itself refute the Orthogonal Vectors Conjecture, a weaker hypothesis used for several polynomial-time problems.

Faster SAT algorithms can also imply new impossibility results. Appendix C uses the algorithms-to-lower-bounds connection developed by Williams. Under the paper’s result, some languages in $\mathbf{E}^{\mathbf{NP}}$ lack certain small Boolean circuits. Here $\mathbf{E}^{\mathbf{NP}}$ denotes problems solvable in deterministic $2^{O(n)}$ time with access to an NP decision oracle, an idealized subroutine answering NP queries. A Boolean circuit is a finite network of gates computing a Boolean function; its size counts gates. This consequence concerns a different class and circuit-size scale, and does not settle P versus NP.

Cryptographic implications require separate parameter and distribution analysis. Worst-case $\mathbf{P} \neq \mathbf{NP}$ alone never guaranteed security. Modern computational security requires hardness on appropriate distributions, against specified adversaries and resource budgets. The paper’s Appendix C.2 derives subexponential inversion for Boolean maps whose output bits each depend on only a fixed number of input bits. It explicitly distinguishes eliminating exponential inversion hardness from eliminating one-wayness against polynomial-time algorithms. No demonstrated break of deployed encryption follows from this tutorial.

Practical optimization gains are an open engineering question. The theorem hides constants and a large-size threshold. A target optimization problem also needs an encoding whose variable growth is acceptable. Domain modeling, data quality, physical experiments, and the cost of verifying a candidate answer remain necessary in real applications.

Conditional lower bounds and alternative assumptions: Section 1.1.3, p. 5. Algorithms to lower bounds: Appendix C.1, pp. 51–52; historical background in Aaronson, pp. 64–72. Local-map inversion: Appendix C.2, pp. 52–53. Worst-case versus cryptographic hardness: Aaronson, pp. 34–37.

Read the video with these corrections in mind

Transcript location	Issue	Precise replacement
00:35–00:51 and 13:13–13:28	ETH is compared with a law of physics.	ETH is a conjecture about worst-case algorithms in a specified computational model. It was not an established physical limit.
02:14–02:22	Aaronson is presented as believing P = NP.	The supplied 2017 survey explicitly argues for P ≠ NP in its abstract and Section 3. It is historical evidence of his stated stance there, not a survey of everyone's current views.
09:18	NP is expanded incorrectly in the supplied transcript.	NP means nondeterministic polynomial time. The verifier definition explains why polynomially solvable problems belong to NP too.
17:29–18:01	The algorithm is described as listening for clues and skipping guesses.	The paper uses exact algebraic sums, shared measurements, interpolation, sparse parity systems, and tensor contraction. Branching alone does not explain its guarantee.
19:32	Efficient is equated with non-exponential.	Polynomial time is narrower than subexponential time. The gap between them is central to this claim.
26:08–27:23	Large constants are linked to simulation speculation.	Impractical constants or thresholds do not logically establish a simulated universe. This is commentary, not a consequence of complexity theory.

The transcript's passages about physics, Yang–Mills theory, local smoothing, academic credit, and another company's result are recorded in the source coverage notes. They are outside the mathematical scope of this ETH tutorial. None is treated as established merely because it appears in the commentary.

12. Problems that test the mechanism

These are pencil-and-paper reasoning problems. Try a question before revealing its solution. No experiments or software setup are required.

▼ **1. Why do unused variables change a count?**

Adjoin two unused declared variables to the running formula. What are the decision answer and count?

Solution. Each of the original two solutions has four extensions, one for each assignment to the two unused bits. The count becomes $2 \cdot 2^2 = 8$. The decision answer remains yes. An exact counter must preserve the declaration of unused variables.

▼ **2. Derive a clause polynomial**

Construct the indicator for $\neg x \vee y \vee \neg z$. On which assignment to its three inputs is it zero?

Solution. The literals have values $1 - X, Y, 1 - Z$, so the polynomial is $1 - X(1 - Y)Z$. Its product term is one exactly when $x = 1, y = 0, z = 1$, the unique case where all literals are false. It is one on every other Boolean assignment.

▼ 3. Which of these runtimes would refute ETH?

Assume a deterministic solver works on every 3-CNF with polynomial input overhead. Compare 1.01^n, $2^{n/1000}$, $2^{\sqrt{n}}$, and $2^{n/\log\log n}$.

Solution. The first two retain fixed positive exponent-to-n ratios: $\log_2(1.01)$ and $1/1000$. Their bounds alone would not refute ETH. The last two have ratios $1/\sqrt{n}$ and $1/\log\log n$, both tending to zero. Either of those guarantees would refute ETH. The formulas are interpreted for sufficiently large n where their logarithms are defined.

▼ 4. Why is $2^{\sqrt{n}}$ above every fixed polynomial?

Solution. Compare with n^c for an arbitrary fixed $c > 0$. Taking base-two logarithms gives $\sqrt{n}$ versus $c\log_2 n$. Their ratio tends to infinity by the exponential-versus-power limit. Hence eventually $2^{\sqrt{n}} > n^c$. This describes the function. It does not prove a lower bound for a problem with that upper bound.

▼ 5. Repair a false statement about NP

“NP problems cannot be solved in polynomial time, but their solutions can be checked quickly.”

Solution. NP problems have polynomial-length yes-certificates verifiable in polynomial time; a no-instance has no accepting certificate. Some NP problems are in P, and every P problem is in NP. Whether every NP problem is in P is precisely the open equality question. Difficulty is not part of the definition of NP.

▼ 6. Follow the reduction arrow

Suppose A reduces to 3-SAT with $O(N)$ variables on inputs of size N. What does a $2^{o(n)}$ 3-SAT algorithm give? What if the guarantee is only $O(N^3)$ variables?

Solution. With polynomial encoding overhead, the first yields $2^{o(N)}$ time for A. In the second case, only $2^{o(N^3)}$ is guaranteed; that need not be subexponential in N. For example, $2^{\sqrt{n}}$ at $n = N^3$ becomes $2^{N^{3/2}}$.

▼ 7. Why do summary collisions preserve multiplicity?

Let a factor depend on bits y, z only through $s = y + z$, with values $f(0), f(1), f(2)$. Sum it over all four assignments.

Solution. The answer is $f(0) + 2f(1) + f(2)$ because there are two assignments with summary one. Replacing the sum by $f(0) + f(1) + f(2)$ would lose multiplicity. The separated routine scans all local assignments in each summary set, so it retains that multiplicity.

▼ 8. Can one linear measurement encode an exponential search?

Explain why $s = \sum_{i=1}^{n} 2^{i-1} x_i$ makes “try every summary” expensive.

Solution. The sum is the binary numeral with bits x_i. Distinct assignments have distinct values ranging from zero to $2^n - 1$. There are 2^n attainable summaries despite only one row. The absolute coefficient sum is $2^n - 1$, exposing the cost that a rank-only argument would hide.

▼ 9. When are Taylor data sufficient?

A polynomial has degree at most five. You know its constant, linear, and quadratic Taylor coefficients at each of two distinct points. Is that enough?

Solution. Yes. The difference between two candidate polynomials would be divisible by $(T-a)^3(T-b)^3$, a polynomial of degree six. A degree-at-most-five nonzero difference cannot have that divisor. Six independent coefficient conditions determine the polynomial.

▼ 10. Recover a coefficient by subset inversion

Let $p(X_1, X_2) = a + bX_1 + cX_2 + dX_1X_2$. Express d using four Boolean evaluations.

Solution. The evaluations are a, $a+b$, $a+c$, and $a+b+c+d$. Thus $p(1,1) - p(1,0) - p(0,1) + p(0,0) = d$. Constant and one-variable terms cancel. This is the two-variable instance of Lemma 3.3's identity.

▼ 11. Why does correction order control degree?

Each correction input is divisible by z. Why can a coefficient of z^j involve at most j correction-variable occurrences?

Solution. Selecting a correction inputs in a product contributes at least z^a. A term with $a > j$ cannot contribute to order z^j. Each selected input is linear in the independent correction vector X, so its X-degree is at most $a \le j$. Boolean multilinearization can only reduce this degree.

▼ 12. Why must sparsification be disjoint for exact counting?

For $F = x \vee y$, add the solution counts of branches $x = 1$ and $y = 1$. Diagnose and repair the result.

Solution. Each branch has two assignments to (x, y), so the sum is four. The original has only three. The assignment $(1, 1)$ was counted twice. Keep the branch $x = 1$ and replace the second with $x = 0, y = 1$: their counts are two and one, respectively, giving three.

▼ 13. Does one parity test characterize satisfaction?

Two violated clauses have violation vector $(1, 1)$. Does the test $v_1 + v_2 = 0$ over $\mathbb{F}_2$ reject it?

Solution. No. The sum is $1 + 1 = 0$ modulo two, so the bad assignment passes. Exactness requires enough carefully chosen checks to reject every nonzero violation vector that arises from an assignment. It is not enough to say that all satisfying assignments pass.

▼ 14. Prove the greedy filter finishes without knowing the true count

Solution. Every candidate accepts all genuine solutions, contributing the same unknown count $\#F$. Minimizing the total count therefore minimizes the bad part. At least one candidate reduces the bad part by a factor at most $3/4$. After $3n$ rounds, at most $2^n(3/4)^{3n} = (27/32)^n < 1$ bad assignments remain. Since the bad part is an integer, it equals zero.

▼ 15. Explain why a residue can be an exact count

Solution. Every solution count lies in $[0, 2^n]$. The modulus 2^{n+1} is larger than every such count, so each has a distinct least nonnegative residue. A congruence computation followed by taking that residue recovers the exact integer. Modulo two alone would preserve only parity and could confuse zero with two.

▼ 16. Why does global truncation beat separate degree bounds?

Solution. The product of a parity indicators separately bounded by degree $O(n)$ could have degree $O(an)$. Instead, every retained product term contains at most $\ell - 1$ copies of the even-valued quadratic h, across all a factors. Together with the at-most-one linear factor per test, its degree in the lifted inputs is at most $a + 2(\ell - 1)$. Composing degree-k inputs gives $k(a + 2(\ell - 1))$, which is $O(n)$ when $a, \ell = O(n)$ and k is fixed.

▼ 17. Why does the sharper recurrence converge?

Solution. Shrinking the wire count by eight multiplies M by $1/8$. Halving the level parameter u multiplies $1/\sqrt{u}$ by $\sqrt{2}$. Their product changes the exponent $M/\sqrt{u}$ by $\sqrt{2}/8 < 1$. The total exponent is bounded by a geometric series rather than accumulating equally at every level. The full theorem still needs its branching, precision, and leaf-work checks.

▼ 18. Interpret the headline without overclaiming

State one consequence of the paper's bound and three things it does not establish.

Solution. If correct, the bound yields deterministic subexponential 3-SAT by counting and testing positivity, refuting ETH. It does not establish P = NP; it does not establish P ≠ NP; and it does not demonstrate a practical break of deployed cryptography. A separate consequence is that conditional lower-bound arguments depending on ETH need their premises reassessed.

13. The paper's turning points: an annotated reading trail

These are the locations to mark in the PDF. Page numbers are one-based viewer pages and match the manuscript's printed pages. Read the question in the third column before opening the page. If you can answer it afterward, the key inference has landed.

Turning point	Exact paper location	The question that reveals the idea	Prerequisite explanation
The claimed refutation in one theorem	Theorem 1.1, p. 2	Why does fixing $k = 3$ and testing positivity contradict a positive exponential rate?	The five-step chain
Count all solutions at once	Section 2.2, equation (2.2), p. 8	Why is a product of clause indicators zero or one, and why is its sum exactly the count?	Logic to polynomial
Move the desired answer to $H(0)$	Section 2.3, equations (2.3)–(2.4), p. 9	Why are non-Boolean intermediate values allowed without changing the target at zero?	Algebraic deformation
Replace global assignments by a small interface	Section 2.4, equation (2.6), p. 10	Why does fixing measurements make group sums multiply, and why is no assignment lost or duplicated?	Shared summaries
Recover information near easy points	Section 2.5, pp. 11–12; Lemma 3.3, p. 24	Why do low-order Taylor coefficients require only small correction subsets?	Taylor corrections and subset inversion
Prove the exponent is	Section 2.6, equations (2.11)–	Where are all three enumeration costs paid, and	The cost ledger

Turning point	Exact paper location	The question that reveals the idea	Prerequisite explanation
sublinear	(2.12), p. 13	why do their exponent ratios tend to zero?	
Construct what the ledger requires	Section 2.7, Steps 2–3, pp. 15–16; Lemmas 4.1–4.3, pp. 28–29	How does one low-degree interpolation polynomial give both few exceptional directions and few mismatches?	The spectral window
See it in four coordinates	Remark 4.4, p. 30	Can you verify the same replacement is local in different groups at points one and two?	Worked identities
Finish the qualitative ETH refutation	Section 2.8, pp. 18–19	Why does the sparse-leaf and variable-copying cost still leave a sublinear exponent for fixed k?	Route A
Keep the stronger route on the original variables	Lemmas 5.1–5.2, pp. 36–37; Lemma 6.2, p. 42	How do modular truncation and deterministic test selection avoid the variable-blowup penalty?	Global truncation and greedy filtering
Obtain the sharpest revised bound	Theorem D.1, p. 53; Lemma D.5, pp. 60–61; equations (D.14)–(D.16), pp. 62–63	Why do weighted crossing terms vanish, and why do the remaining contractions shrink fast enough?	The recursive argument
Read the implications without reversing reductions	Section 1.1.3, p. 5; Appendix C, pp. 51–53	Which target algorithms are actually constructed, and which lower-bound premises merely disappear?	Consequences and limits

The central realization. An exponential number of assignments need not force an algorithm to visit each assignment. The paper claims an exact algebraic way to aggregate them, recover the

answer from structured evaluations, and prove that the combined overhead has a sublinear exponent. Generality and the full cost ledger are what turn that idea into an ETH refutation.

14. Source coverage and how to continue

The five original files and the subsequently supplied Samuel transcript were inventoried. Both PDFs were read in full as extracted text across author and independent editor; nine mathematical checkpoint pages in the OpenAI paper were also rendered and visually inspected. The screenshot and both full transcripts were inspected. Reading a bibliography does not mean that every cited work was read. The videos were available as supplied text only; no full audio or visual review is claimed. Original source files remain local; public links below open the original publications and videos. Every `*.txt` file is excluded from the repository.

Supplied source	Identity and coverage	Use and boundary
paper.pdf	OpenAI, *The Exponential Time Hypothesis is False*, dated 17 October, revised 25 October 2026; 67 pages, including appendices and references.	Primary mathematical claim and exact proof map. Local bytes matched the dated PDF in OpenAI's repository by SHA-256 on 27 October 2026. Full textual reading and selected rendered-page inspection; no complete proof certification.
pnp.pdf	Scott Aaronson, *P ?= NP*, a survey of the field in 2017; 122 pages including bibliography and glossary.	Definitions, reductions, counting, historical ETH framing, proof barriers, and algorithms-to-lower-bounds context. Full text read by the independent source editor. Its historical frontier is not silently promoted to 2026 status.
Wes Roth's video	Supplied commentary transcript, through approximately 32:55, followed by unrelated recommendation text.	Motivation, timestamp navigation, and a correction ledger. Names and technical language have transcription errors. Its physics and sociology passages are outside this tutorial's scope. The transcript is kept local.
Samuel Allen Alexander's	Subsequently supplied commentary transcript,	Discussion of the paper introductions, reduction-size warnings, differing runtime

Supplied source	Identity and coverage	Use and boundary
video	through approximately 34:04, followed by unrelated recommendation text.	bounds, and practical limits. His comparison with another paper and priority discussion remain commentary, rather than separately verified research claims. The transcript is kept local.
ETH.png — original announcement	Screenshot excerpt describing ETH, Lean, and an external expert group.	Announcement evidence, checked against the live primary announcement. It is not itself a proof. The screenshot stays local.
README.txt — excluded	Empty file, zero bytes.	Inventoried; contains no substantive source material.

The independent source notes are in the Aaronson reading brief. A per-source analysis and transcript coverage map, source manifest, and claim and dependency ledger preserve the distinction between source statements, checked elementary derivations, and explanatory synthesis. Computed example results check the teaching arithmetic; they do not reproduce the research algorithm.

How your lecture-note workflow was adapted

The source-analysis and rigorous-explainer skills supplied provenance, first-use definitions, explicit derivations, and local validation. Your transcript-grounded textbook workflow and existing MIT lecture-note repository supplied the model of an offline reader, worked cases, full solutions, notation discipline, and an independent scientific-editor pass. This is a standalone tutorial; its order follows your foundations-first preference rather than the commentary video's order. It is not an extension or endorsement by NPTEL, Stanford, MIT, or the paper's authors.

For further theoretical study, the most useful next step is to reconstruct the separation identity and the subset-inversion argument without looking. They are small enough to master completely and are the bridge from elementary Boolean reasoning to the paper's machinery. After that, read the four-coordinate example alongside the spectral construction. Save Appendix D's full precision proof for a second pass.

Two connections worth pursuing after the foundations

Counting as a partition function. A statistical-mechanical partition function sums weights over configurations. The SAT count is the special case where allowed configurations have weight one and forbidden configurations have weight zero. The tensor-network representation makes this formal analogy

visible. The claimed complexity result is for its specified Boolean networks and arithmetic model; it does not automatically make physical partition functions easy.

Efficient algorithms as evidence for lower bounds. Faster satisfiability can be used inside a contradiction argument about hypothetical small circuits. That reverses the common intuition that faster algorithms only weaken hardness statements. Compare Appendix C with Aaronson's Section 6.4 after you are comfortable with reductions. The precise resource classes are the substance of that connection.

Reminder for later: hands-on experiments are deferred at your request. Once you can explain the three-cost ledger and why $H(0)$ is recoverable, revisit a tiny SAT counter, Boolean-to-polynomial identities, and a small Hermite reconstruction. Those would illustrate the foundations; they would not reproduce the new research solver. No timed reminder has been scheduled.

Independent study tutorial prepared 27 October 2026. Original teaching examples are distinguished from cited research claims. All mathematics rendering is bundled locally, with its license. Source repository · Live GitHub Pages edition.